LF AI & DATA

Security and Compliance Working Group

AI SAFETY BEST PRACTICES FOR REGULATED ENVIRONMENTS

A Comprehensive Guide to AI Safety Risks Across AI Lifecycles and AI Stack

Version 1.1 | September 2026



Table of Contents

About the Authors 5
Executive Summary 7
1 Introduction and Scope 10
1.1 Purpose and Scope 10
1.2 Target Audience 11
1.3 Out of Scope: Traditional Cybersecurity 11
1.4 Document Structure 13
1.5 Status: Publish 13
2 AI Compliance Landscape 13
2.1 Current Trends in AI Governance and Compliance 14
2.2 Legal and Regulatory Frameworks 14
2.2.1 NIST AI Risk Management Framework 14
2.2.2 ISO/IEC 42001: Artificial Intelligence Management Systems 15
2.2.3 EU Artificial Intelligence Act 16
2.2.4 National Regulatory Frameworks 16
3 AI Alliance Risk Atlas Nexus 17
3.1 AI Alliance Risk Atlas Nexus 17
3.2 Risk Atlas Nexus Overview 18
4 AI Safety Across the AI Stack 19
4.1 AI Stack Architecture Overview 19
4.2 Comparison with CNCF Technology Radar 19
4.3 Layer-by-Layer Drill-down Safety Analysis 19
Layer 7: Applications and APIs 19
Layer 6: ML Orchestration 20
Layer 5: Model Frameworks and Inference Serving 21
Layer 4: Data Pipeline and Orchestration 22
Layer 3: Platform Service (Governance, Observability, Security, Cost) 23
Layers 2-1: Infrastructure (out of scope for AI but worth noting) 24
5 AI Safety Across the Model Lifecycle 24
5.1 Data Collection and Curation 24
5.2 Model Development and Training 25

5.3 Model Evaluation and Testing 26
5.4 Model Deployment 27
5.5 AI Operations and Monitoring 28
5.6 Model Updating and Retraining 29
6 AI Safety Across the Agentic Lifecycle 30
6.1 Overview 30
6.2 Agent Design and Planning 30
6.3 Agent Development and Integration 31
6.4 Agent Testing and Validation 32
6.5 Agent Deployment 33
6.6 Agent Operations and Monitoring 33
6.7 Agent Updating and Retraining 34
6.8 Multi-Agent Coordination and Safety 35
6.9 Component Reference: Tools and OSCAL Mappings 36
7 Three-Phase Methodology 37
7.1 Phase 1: AI Risk Mapping 38
7.2 Phase 2: Human-in-the-loop Rule Derivation 39
7.2.1 Traditional approach to rules and metrics definition - tedious, manual, and time consuming 39
7.2.2 Human-in-the-loop rule derivation - automation using GenAI 40
7.3 Phase 3: Validation & Automation 43
8 Actionable Machine-Readable Compliance with OSCAL 44
8.1 Introduction to NIST OSCAL v1.2.2 44
8.2 From Documentation to Automation 45
8.3 OSCAL Component Definition Example 45
8.4 Continuous Compliance Verification 46
9 Implementation - Next Steps 47
9.1 From Compliance Framework to Policy Enforcement 47
9.2 Policy Engine Landscape for AI 47
9.3 Mapping Rules to Policy Engines: Practical Examples 48
9.3.1 Infrastructure and Platform Compliance Checks (OPA, Kyverno) 48
9.3.2 Infrastructure-as-Code Compliance Checks (Terraform Sentinel, Checkov) 49
9.3.3 Application-Level Compliance Checks (OPA in Application Context) 50

9.3.4 AI Inference Guardrails Compliance Checks (NeMo, Guardrails AI, Lakera Guard) 51
9.4 Model Cards, Agent Cards, and Compliance Rule Relationships 53
9.4.1 Model Cards: Compliance Evidence Documentation 53
9.4.2 Agent Cards: Extending Documentation for Agentic Systems 54
9.4.3 Integrating Cards into Compliance Workflows 55
10 Future Directions and Emerging Challenges 56
10.1 Agentic AI Systems 56
10.2 Regulatory Evolution 56
10.3 Emerging Threat Vectors 57
11 Conclusion 57
11.1 Summary of Key Practices 57
11.2 Organizational Capabilities 57
11.3 Call to Action 58
References and Resources 59
Appendix A: Quick Start Guide 60
Phase 1: Assessment and Planning (Weeks 1-2) 61
Phase 2: Pilot Implementation (Weeks 3-6) 63
Phase 3: Expand Coverage (Weeks 7-12) 64
Phase 4: Operationalize and Scale (Weeks 13-16) 65
Success Criteria and Metrics 66
Common Pitfalls to Avoid 67
Next Steps After Quick Start 67
Appendix B: Tool Ecosystem and Resources 69
Policy Engines 69
AI Stack Tools by Layer 71
Standards and Compliance Frameworks 74
Model Documentation Standards 75
Community Resources and Working Groups 76

About the Authors

This white paper was developed through collaborative efforts of the LF AI & Data Foundation Risk and Compliance Subgroup, with contributions from practitioners across industry, academia, and standards organizations. The subgroup brings together expertise in AI/ML engineering, security, compliance, and governance to advance best practices for trustworthy AI systems.

For questions, comments, or contributions to future revisions, contact the LF AI & Data Security and Compliance working group: https://github.com/lfai/security-and-compliance

Authors:

- Anca Sailer, IBM
- Vikas Agarwal, IBM
- Thalia Hooker, Red Hat
- Nitin Shetty, IBM
- Uman Ahmed Mohammed, IBM
- Deniza Beslinger, IBM
- Vinodkumar R Nair, IBM
- Vincent Caldeira, Red Hat
- Robert Ficcaglia, SunStone Secure
- Lou DeGenaro, IBM

Contributors:

- Brittany Huntzberry, Red Hat
- David Kliemann, IBM
- Imran Khan, IBM
- Inge Vejsbjerg, IBM
- Matheus Caldeira Matias, IBM
- Matthew Hagen, Red Hat
- Rubayat Khan, IBM
- Sergiu Macovei, IBM
- Sreelatha Sankaranarayanan, IBM
- Swarnalata Patel, IBM
- Tamara Reyes, IBM
- Victoria Baker, IBM

Acknowledgement

We gratefully acknowledge the contributions of the following individuals for their valuable insights and thoughtful discussions that informed this white paper:

- Anne Leslie, IBM
- Aparna Kurada, IBM
- Evaline Ju, IBM
- Matt Rutkowski, LF AI & Data, Security & Compliance WG
- Monica Sasso, Red Hat
- Rob Geada, Red Hat
- Robert-Igor Bartsch

We gratefully acknowledge the executive sponsors from IBM and Red Hat for their leadership and support in making this white paper possible:

- Priya Nagpurkar, IBM
- Shuchi Sharma, Red Hat

Executive Summary

The rapid proliferation of artificial intelligence (AI) systems across critical domains such as healthcare, finance, and autonomous transport that traditional cybersecurity frameworks were not designed to address. Established frameworks like NIST SP 800-53 (US), ISO/IEC 27001/27002 (Europe), ISM/CSA/MAS (AP), address infrastructure security, but organizations deploying AI systems face the added complexity of ensuring the AI itself behaves safely, fairly, transparently, and reliably.

AI safety risks are multifaceted and encompass a broad spectrum of vulnerabilities:

- **Traditional AI risks** (e.g., data bias, lack of transparency) lead to overfitted outcomes.
- **Amplified by generative AI**, these risks result in training data contamination and unexplainable outputs.
- **Specific to generative AI** are harmful outputs (hallucinations, false confidence) and unintended non-deterministic behaviors in agentic systems.
- **Amplified by synthetic data** are overfitting and unrepresentative data distributions.
- **Specific to synthetic data** is the risk of lacking domain expertise, complicating safe model weight development.

Managing these risks requires specialized practices throughout the AI lifecycle. This paper introduces a structured methodology using two complementary lenses:

1. **AI Lifecycle:** Mapping risk controls from data collection through model serving, inference orchestration, and agentic orchestration.
2. **AI Technical Stack:** Identifying safety measures from infrastructure through the AI platform, agent framework, and application layers.

This dual perspective ensures comprehensive coverage, allowing product and operation teams to focus on their specific control points.

Our guidance draws from established taxonomies and frameworks, including the AI Alliance Risk Atlas Nexus, the NIST AI Risk Management Framework (Govern-Map-Measure-Manage), ISO/IEC 42001, and real-world implementations from the Cloud Native Computing Foundation (CNCF) and LF AI Foundation. We provide actionable guidance with concrete examples (e.g. compliance-as-code machine readable artifacts), such as using HuggingFace model cards and MLflow for bias and fairness tracking, Ray Train/Serve and KServe for distributed patterns and monitoring, and LangChain/LangGraph flows with prompt validation for application developers.

The evolving regulatory landscape, including the EU Artificial Intelligence Act (risk-based requirements, substantial penalties), the gaining traction of the NIST AI Risk Management Framework in the US, and the certification offered by ISO/IEC 42001,

creates a complex patchwork of requirements. Our best practices, derived from the NIST AI Risk Management Framework, offer broad applicability across other standards due to control overlap. Given this overlap, this guidance can be applied to other standards through controls' crosswalk mappings (see OSCAL Mapping Best Practices white paper for methodology).

A key innovation is the adoption of machine-readable representation through the NIST compliance-as-code standard OSCAL (Open Security Controls Assessment Language), specifically its Component Definitions model. Encoding safety requirements in a standardized, executable format that is version-controlled alongside infrastructure-as-code and AI governance moves organizations beyond static documentation to dynamic, continually verifiable controls.

Adopting this modern code-based DevSecOps methodology, this approach enables rapid iteration. Crucially, it upholds rigorous safety and compliance standards, maintaining these postures even at runtime inference speed to guarantee a fast user experience. Compliance checks can be integrated into CI/CD and AI pipelines to prevent non-compliant configurations from reaching production during deployment. Furthermore, continuous verification is achieved through integration with runtime policy and AI safety engines, which can immediately alert or enforce policy when operational conditions lead to compliance drift. Additionally, using OSCAL standard and the CNCF Compliance-to-Policy tooling, practitioners can reuse their current cybersecurity and resilience governance to seemingly upgrade from integration with traditional policy engines (e.g., Kyverno, OPA, Ansible, Terraform) to integration with AI specific guardrail policy engines (e.g., Purple Llama, PyRIT for model training, Guardrails AI, NeMo for agentic workloads).

This white paper serves a wide range of audiences:

- **AI/ML Engineers** need guidance on bias detection and mitigation as they develop models.
- **MLOps/DevOps Professionals** require continuous monitoring strategies.
- **Compliance Officers** need structured frameworks for mapping AI systems to mandated regulatory requirements.
- **Data Scientists** must ensure data privacy and fairness as they curate datasets.
- **Product Managers** need to understand risk tradeoffs.
- **Executives** require a strategic perspective on building trustworthy AI capabilities aligned with regulations.

Each audience will find guidance and actionable compliance policy artifacts, referenced in the CNCF Project OSCAL-Compass git repository, appropriate to their role and to the specific selected projects classified by the AI lifecycle and technical stack as illustrated in Figure 1.

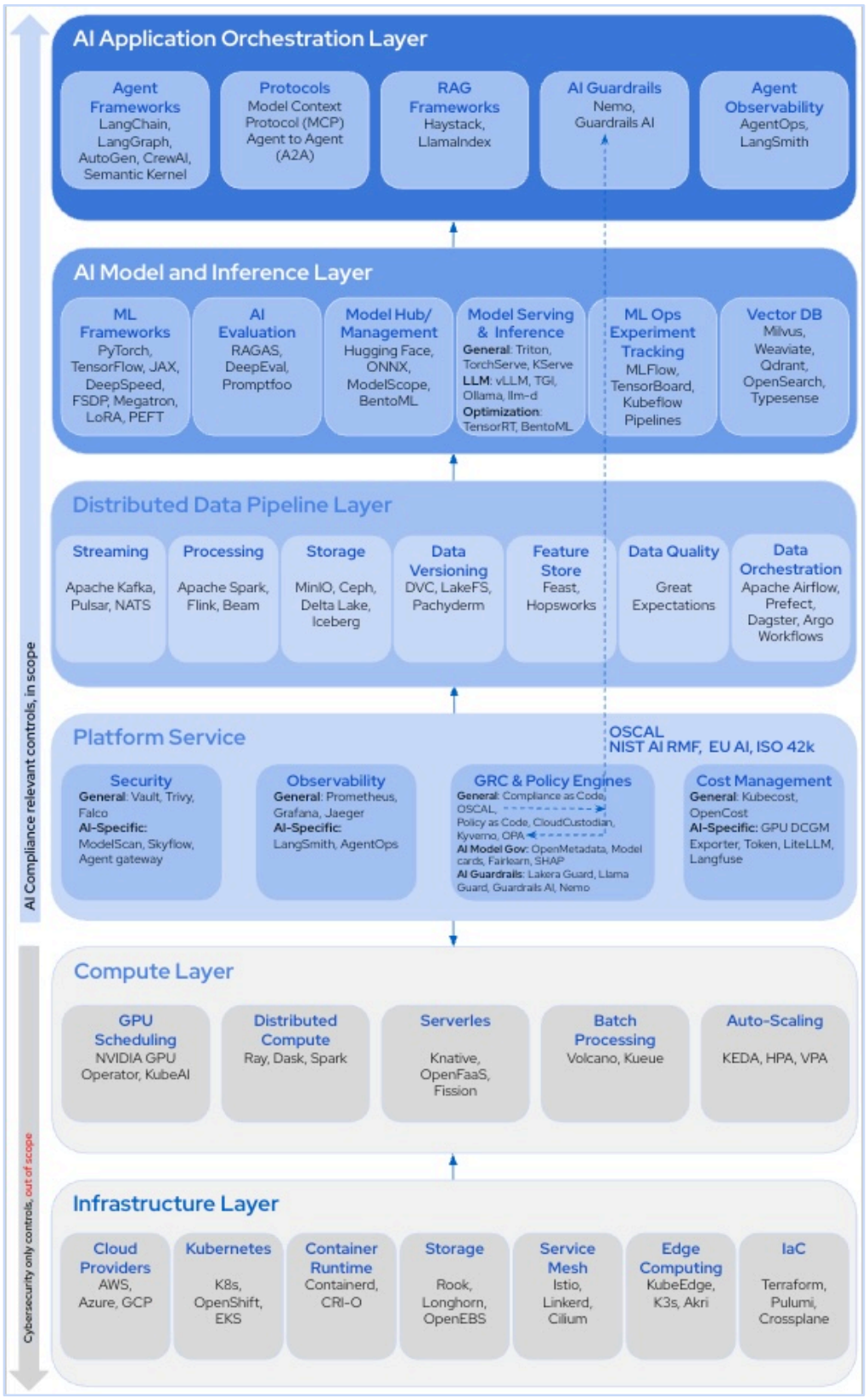


Figure 1: Open-Source AI Technology Stack with Compliance Controls

1 Introduction and Scope

This white paper provides a comprehensive guide for organizations seeking to implement robust AI safety practices across their AI systems.

1.1 Purpose and Scope

Our primary objectives are threefold:

1. **Establish common understanding** of AI-specific safety risks that extend beyond traditional software cybersecurity.
2. **Provide actionable machine-readable compliance-as-code artifacts** to support the governance and audit of both AI development lifecycle and underlying technical stack.
3. **Demonstrate automated AI safety verification at scale** by transitioning from static PDF guidelines to machine-readable compliance artifacts.

The paper covers both the technical and governance aspects of AI safety.

Technical Scope: While both AI safety and traditional cybersecurity are essential for system trustworthiness, this paper focuses exclusively on **AI safety risks**, recognizing that traditional cybersecurity is a mature and readily applicable field. Our focus includes:

- NIST AI RMF concerns, such as data quality policies ensuring unbiased and representative training data.
- Model fairness validation to detect discrimination pre-deployment.
- Inference and agentic output reliability verification to prevent user-facing hallucinations.
- Operational monitoring to maintain system behavior as real-world conditions change.

Governance Scope: We address the methodology for generating AI safety requirements and responsibilities, along with standardizing policy representation. This is specifically aimed at automating compliance and increasing the efficiency of manual labor, given the industry's need for continuous compliance amid increasing regulations.

The discussion is framed around **cloud-native AI architectures and tooling**. This reflects the modern trend of AI systems relying on containerized, agentic workloads orchestrated across distributed infrastructure, utilizing ephemeral compute resources for cost optimization, and employing distributed data pipelines to meet performance and regulatory needs across multiple systems and geographic locations.

1.2 Target Audience

This white paper addresses multiple stakeholder groups involved in AI system development, deployment, and governance. Each group brings different perspectives, priorities, and constraints to AI safety, and effective safety programs require coordination across all these stakeholders.

Compliance Officers and Risk Managers tasked with ensuring regulatory adherence will find structured frameworks for mapping AI systems to regulatory requirements. This includes NIST AI RMF function implementation across Govern-Map-Measure-Manage, potentially mapped to EU AI Act risk categorization determining which systems qualify as high-risk requiring strict controls, ISO/IEC 42001 management system requirements supporting certification, and industry-specific AI regulations emerging in healthcare, financial services, and other heavily regulated sectors. This paper helps these professionals translate often vague regulatory language into specific technical and organizational requirements, verifiable then through auditing and assessment that they are met.

MLOps and DevOps Professionals responsible for deploying and operating AI systems need to understand operational practices for model monitoring including drift detection comparing production input distributions to training distributions, continuous fairness assessment across demographic subgroups to detect degradation before it causes widespread harm, and incident response protocols specific to AI safety issues.

Data Scientists and Analysts preparing training datasets need guidance on sampling strategies ensuring representative coverage of populations and scenarios, bias detection techniques identifying systematic imbalances or quality issues, mitigation approaches correcting identified problems through re-sampling or data augmentation, provenance documentation tracking data sources and transformations, and privacy-preserving methods protecting sensitive information while maintaining data utility.

AI Application and Agent Developers implementing agentic systems need guidance on AI safety including prompt validation, tool authorization, output filtering, and integration with AI dedicated policy engines to prevent AI-specific attacks like prompt injection and context poisoning within their orchestration frameworks.

1.3 Out of Scope: Traditional Cybersecurity

Traditional cybersecurity concerns are explicitly out of scope for this white paper, as they are comprehensively addressed by established frameworks and require different expertise than AI safety. However, practitioners must address these concerns through appropriate cybersecurity frameworks in parallel with AI safety efforts.

The following areas are explicitly out of scope, and the relevant established frameworks are noted:

- **Infrastructure Security:** Covered by frameworks such as ISO 27001 and NIST SP 800-53 (Physical and Environmental Protection family). This includes physical security controls for data centers, physical access management and badge systems, hardware supply chain verification, tamper resistance, and environmental controls (temperature, humidity, and power).
- **Network Security:** Addressed by NIST SP 800-53 (System and Communications Protection family) and industry standards (e.g., PCI DSS). This encompasses firewalls, network perimeter controls, intrusion detection and prevention systems, network segmentation, traffic analysis, DDoS protection, and secure network protocols.
- **Access Control Mechanisms:** Managed by NIST SP 800-53 (Access Control and Identification and Authentication families). This involves authentication (including multi-factor), authorization (role-based access control), identity management, privilege escalation prevention, session management, and credential/password policies.
- **Encryption Standards:** Covered by NIST SP 800-53 (System and Communications Protection family) and specific NIST cryptographic standards. This includes data at rest encryption (AES), data in transit protection (TLS/SSL), key management, cryptographic algorithm selection, certificate management (PKI), and quantum-resistant cryptography.
- **Vulnerability Management:** Addressed by NIST SP 800-53 (Risk Assessment and System and Information Integrity families). This covers security patching, vulnerability scanning and assessment, penetration testing, threat modeling, secure software development lifecycle (SSDL) practices, and third-party security assessments.

Organizations deploying AI systems must treat AI safety and traditional cybersecurity as complementary domains, addressing both simultaneously.

Resources for leveraging cybersecurity in the context of AI include:

- Model Content Protocol (MCP) Security
- OASIS CyberSec for AI projects
- CNCF CyberSec Controls Catalog for CNCF projects

AI-specific misuse risks — deepfakes, AI-generated fraud at scale, autonomous cyberweapons — occupy a gray zone between AI safety and traditional cybersecurity. While out of scope here, organizations should reference:

- MITRE ATLAS (v5.4.0, updated May 2026): https://atlas.mitre.org
- OWASP Top 10 for Agentic Applications 2026: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
- OWASP Agentic Security Initiative: https://genai.owasp.org/initiatives/agentic-security-initiative/

1.4 Document Structure

This white paper is structured to offer a comprehensive look at AI safety, encompassing both the regulatory landscape and technical best practices.

Key Chapters:

- **Chapter 2: AI Compliance Landscape:** Explores legal and regulatory frameworks, international standards, and industry guidelines from various stakeholders.
- **Chapter 3: AI Alliance Risk Atlas Taxonomy:** Provides a structured understanding of threat origins and how they are addressed by existing regulation controls.
- **Chapter 4: AI Safety Across the Stack:** Analyzes each technical layer—from infrastructure to applications—detailing specific best practice policies.
- **Chapter 5: AI Safety Across the Lifecycle:** Addresses safety considerations for every phase, from data collection through ongoing operations.
- **Chapter 6: AI Safety Across the Agentic Lifecycle:** Provides lifecycle-specific guidance for safely developing, deploying, and operating agentic AI systems.
- **Chapter 7: Three-Phase Methodology:** Outlines a three-phased approach for achieving continuous, automated AI governance.
- **Chapter 8: Machine-Readable Compliance:** Introduces the NIST OSCAL Component Definition model as an enabler for automated AI policy governance.
- **Chapter 9: Implementation – Next Steps:** Details the critical next step of translating compliance requirements into enforceable policies at both development and runtime.
- **Chapter 10: Future Directions:** Discusses emerging threats and systems, including agentic AI.
- **Chapter 11: Conclusion:** Offers a synthesis of the paper and a call to action.

1.5 Status: Publish

2 AI Compliance Landscape

This chapter provides a critical overview of the most significant current trends and legal and regulatory frameworks shaping the responsible development and deployment of AI systems globally. From the foundational guidance of the voluntary NIST AI Risk Management Framework to the stringent, mandatory requirements of the EU Artificial Intelligence Act and emerging national regulations in Asia and the Americas, organizations must navigate a complex, multi-jurisdictional environment. Understanding these frameworks—including the key functions of NIST RMF (Govern, Map, Measure, Manage) and the risk-based classification of the EU AI Act—is essential not only for

legal compliance and avoiding substantial penalties but also for establishing robust, ethical, and trustworthy AI management systems.

2.1 Current Trends in AI Governance and Compliance

The global response to AI governance is characterized by diverse regulatory frameworks, proposed legislation, and guidance that vary significantly in scope, stringency, and enforcement across countries. This variation complicates compliance for organizations operating internationally, necessitating sophisticated strategies to meet the strictest requirements efficiently and avoid redundant efforts.

Key governmental and international bodies, including NIST in the U.S., national AI councils, and the OECD, have issued various AI governance frameworks and guidance. Notably, the NIST AI Risk Management Framework has emerged as a de facto standard in the United States.

2.2 Legal and Regulatory Frameworks

Organizations must deeply understand the complex global regulatory environment surrounding AI, which includes enforceable laws with significant penalties, pending legislation, and non-binding frameworks that often foreshadow future mandatory rules. This understanding is critical because failure to comply with applicable regulations can lead to substantial fines, operational restrictions, and harm to the organization's reputation.

2.2.1 NIST AI Risk Management Framework

The NIST AI Risk Management Framework, published in January 2023 by the U.S. National Institute of Standards and Technology, is rapidly becoming the de facto standard for voluntary AI risk management in the United States. Flexible and technology-neutral, the framework offers high-level principles adaptable to all organizations, although implementing concrete rules remains a burden for practitioners.

The framework is structured around four core functions Govern-Map-Measure-Manage that parallel the NIST Cybersecurity Framework's five functions while addressing AI-specific concerns. These functions are not sequential steps but interconnected and iterative processes that should be applied throughout the AI system lifecycle to effectively manage risks.

- **Govern** establishes the organizational foundation, covering legal, ethical, and stakeholder aspects.
- **Map** identifies and contextualizes risks by considering system use, affected parties, and potential harms.
- **Measure** assesses and tracks risks using quantitative and qualitative approaches, including establishing metrics, testing, monitoring inputs, and

tracking performance across populations.
- **Manage** focuses on prioritizing and responding to risks through mitigation strategies, technical controls, procedural safeguards, and incident response planning.

While NIST AI RMF provides organizations with structured guidance for addressing AI risks and a common language for discussing risks with stakeholders, it lacks the specific procedures and metrics that are needed to implement it. Our goal is to address that gap between the high-level guidance and the technology specific policies and metrics as detailed in Chapters 4 and 5.

2.2.2 ISO/IEC 42001: Artificial Intelligence Management Systems

ISO/IEC 42001, published December 2023, is the international standard for organizational AI management systems. It sets requirements for establishing, maintaining, and continually improving AI management systems, focusing on ethical and responsible AI development and use, including risk management, data governance, security, transparency, and stakeholder engagement.

Based on Annex SL, the standard's common framework allows integration with existing ISO certifications (e.g., ISO 27001, ISO 9001), reducing effort and enabling holistic governance. Certification to ISO/IEC 42001 is now a competitive advantage for organizations demonstrating trustworthy AI to customers, regulators, and partners.

NIST has published an official crosswalk mapping all AI RMF subcategories to ISO 42001 clauses (https://airc.nist.gov/docs/NIST_AI_RMF_to_ISO_IEC_42001_Crosswalk.pdf). A machine-readable OSCAL Mapping JSON is available in the CNCF Sandbox OSCAL Compass content repo.

The key functional mappings are:

Table 1: NIST AI RMF to ISO42001 mapping

NIST AI RMF Function	ISO 42001 Equivalent	Example Controls
GOVERN	Clauses 4-5, B.2-B.3	4.1 Context, 5.2 AI Policy, B.3.2 Roles
MAP	Clauses 6.1, B.5	6.1.4 Impact assessment, B.5.2 Process

MEASURE	Clauses 8-9, B.6-B.7	B.6.2.4 Verification, 9.1 Monitoring
MANAGE	Clauses 8.3, 10, B.8-B.10	8.3 Risk treatment, 10.2 Corrective action

2.2.3 EU Artificial Intelligence Act

The EU Artificial Intelligence Act is the world's most comprehensive and stringent AI regulatory framework, utilizing a risk-based approach to ensure AI systems in the EU are safe, transparent, non-discriminatory, and uphold fundamental rights. It classifies AI into four tiers:

1. **Unacceptable Risk:** Prohibited outright (e.g., social scoring, real-time public biometric identification with limited exceptions).
2. **High-Risk:** Subject to strict requirements, conformity assessments (internal or third-party), and registration in an EU database before market placement. This includes AI used as safety components (e.g., medical devices) and in sensitive domains (e.g., critical infrastructure). Providers face substantial penalties for non-compliance (up to €35M or 7% of worldwide annual turnover).
3. **Limited-Risk:** Requires transparency obligations, such as disclosing when users interact with a chatbot or view deepfake content.
4. **Minimal-Risk:** Faces no specific requirements under the Act, covering most applications and preserving regulatory focus for higher-risk systems.

2.2.4 National Regulatory Frameworks

South Korea's AI Basic Act (Jan 2026) is the second comprehensive AI law globally, aiming to foster innovation and responsible AI through ethical and regulatory standards, building public trust.

Canada's proposed Artificial Intelligence and Data Act (AIDA) (Bill C-27) sought to regulate responsible AI development, especially for high-impact systems, ensuring fairness, safety, and accountability, but did not pass initially.

Brazil's Bill No. 2338/2023 proposes comprehensive AI regulation focusing on transparency, security, and rights protection, including risk classification, algorithmic impact assessments, transparency obligations, and human oversight, to be overseen by an AI regulatory authority.

Australia's Voluntary AI Safety Standard offers ten non-mandatory guardrails for organizations, covering accountability, human oversight, transparency, fairness, privacy, security, reliability, safety, contestability, risk management, data governance, and stakeholder engagement.

All these regulatory frameworks overlap significantly with the NIST AI RMF, focusing on AI risk and safety. Mapping them to the NIST AI RMF, as instructed in the OSCAL Mapping Best Practices white paper and exemplified in the NIST crosswalk mapping of AI RMF subcategories to ISO 42001 clauses (https://airc.nist.gov/docs/NIST_AI_RMF_to_ISO_IEC_42001_Crosswalk.pdf or machine-readable CNCF Sandbox OSCAL Compass content) simplifies implementation by identifying common requirements needing only delta implementation for additional requirements not covered by NIST AI RMF.

3 AI Alliance Risk Atlas Nexus

This chapter introduces the **Risk Atlas Nexus**, a comprehensive framework designed to describe, categorize, and manage the inherent safety risks associated with AI systems. The framework employs a standards-based ontology and incorporates taxonomies from leading sources, including the AI Alliance Risk Atlas, NIST AI RMF, and OWASP Top 10 for LLMs, to provide a holistic view of potential threats. It outlines key risk categories—covering concerns such as Bias and Fairness, Hallucinations, Model Robustness, and Ethical and Societal Risks—organized by the threat's origin: Input, Inference, Output, Non-Technical, and Use Context Risks. The tooling aims to standardize AI governance by providing actionable, open-source infrastructure for organizations to assess risk severity (including EU AI Act classification) and map generic risks and controls to specific, enforceable policy rules. An additional comprehensive framework designed to describe, categorize, and manage the AI safety risks is the MIT AI Risk taxonomy.

3.1 AI Alliance Risk Atlas Nexus

IBM Research donated the Risk Atlas Nexus to the AI Alliance and shifted it to a community-driven approach for linking and curating AI risk resources. This comprehensive catalogue helps practitioners understand risks in traditional and generative AI, differentiating risks across paradigms like data, model behavior, security, operational, and ethical/societal categories. It maps its risk taxonomy to external threat taxonomies, such as OWASP Top 10, linking technical threats (e.g., prompt injection) to broader organizational risks (e.g., brand reputation).

Key features include standards-based ontology structure to describe AI system risks and models, ingestion of taxonomies of AI risks (e.g., AI Alliance Risk Atlas, IBM Granite Guardian, MIT AI Risk Repository, NIST AI RMF Generative AI Profile, AI Risk Taxonomy (AIR 2024), AILuminate Benchmark, Credo's Unified Control Framework, and OWASP Top 10 for Large Language Model Applications), risk categorization including EU AI Act classification, and reporting capabilities (YAML, OWL, JSON) for integration with other tools. The tooling aims to lower barriers to implementing AI governance by providing open-source infrastructure that organizations of all sizes can adopt and adapt.

Our methodology goes one step farther by additionally mapping the generic high-level risks-controls association to targeted products and open-source projects' specific AI policy rules directly actionable in policy enforcement engines.

3.2 Risk Atlas Nexus Overview

The AI safety risks are structured within the Risk Atlas into five categories, which are defined by the source of the threat: Input Risks, Inference Risks, Output Risks, Non-Technical Risks, and Use Context Risks. Following are risk examples by category:

- **Input Risks:** Data bias, copyright exposure, unverified sources, prompt injection, and malicious manipulation.
- **Inference Risks:** Hallucinations, emergent behavior, untraceable influence, model drift, and inconsistent outputs.
- **Output Risks:** Fidelity issues, harmful/discriminatory content, misinformation, and privacy leakage.
- **Non-Technical Risks:** Unclear provenance, inadequate documentation/governance, insufficient oversight, and lack of accountability.
- **Use Context Risks:** Misuse, societal harm, trust erosion, job displacement, and environmental costs.

These relate to key AI concerns:

- **Bias and Fairness:** Discriminatory outcomes, higher error rates for minority groups, and underrepresentation in training data leading to poor performance for those groups.
- **Transparency and Explainability:** Black-box decision-making, lack of clear rationales, and the inability to trace bias or audit stochastic solutions without clear supply chains and reasoning traces.
- **Hallucinations and Output Fidelity:** Factually incorrect, plausible-sounding outputs presented with false confidence, misleading users, generating misinformation, and unreliable inference.
- **Data Quality and Provenance:** Dataset bias, unverified training sources, copyright risks, and poor documentation preventing fitness assessment.
- **Model Robustness and Reliability:** Performance degradation (model drift) over time, disproportionate performance drops for specific groups, and sensitivity to minor input changes.
- **Ethical and Societal Risks:** Potential for harm in critical domains (employment, credit, justice), erosion of public trust, large-scale misinformation generation, and disproportionate job displacement.

For more comprehensive details visit the taxonomy official page at Risk Atlas Nexus.

4 AI Safety Across the AI Stack

This chapter delves into the foundational concept of AI safety applied across the entire technology infrastructure—the AI Stack. Modern AI systems are complex, layered architectures, and a comprehensive approach to managing trustworthiness, fairness, transparency, and reliability requires analyzing the specific risks inherent at each level, from the foundational compute and platform services to the end-user applications, agents, and APIs. This chapter systematically breaks down the AI stack to identify common safety risks and provides tangible examples of best practices and tool components that can be used to mitigate these risks at the point of origin, ensuring robust and compliant AI systems from development through deployment.

4.1 AI Stack Architecture Overview

As depicted in Figure 1, the AI layered architecture extends from the physical infrastructure up through AI-specialized frameworks and AI applications, representing a safety perspective distinct from traditional cybersecurity concerns.

4.2 Comparison with CNCF Technology Radar

The CNCF Technology Radar (October 2025), provides a distinct, tool-focused view on several areas: AI Inferencing, ML Orchestration, and Agentic AI Platforms. It surveys developer experiences, categorizing AI technologies by **adoption readiness** (Adopt, Trial, Assess, Hold) and **use case** (Inferencing, Orchestration, Agentic AI).

This white paper and the CNCF Radar are complementary resources. The Radar provides **specific tool recommendations** based on community experience and tracked technology maturity, updating quarterly. In contrast, this white paper focuses on showcasing **AI safety policies** and their compliance mapping across the entire AI stack and lifecycle, emphasizing comprehensive coverage and policy management rather than specific tool selection.

Note: Use the CNCF Radar for **tool selection** within a technology layer and use this framework for **identifying the AI safety best practices** and policies for those technologies.

4.3 Layer-by-Layer Drill-down Safety Analysis

Layer 7: Applications and APIs

Description: User-facing interfaces and agent frameworks that compose multiple AI capabilities, manage multi-step reasoning, and coordinate tool access for autonomous systems.

Safety Risks:

- Prompt injection/Indirect prompt injection
- Output validation failure (hallucinations)
- Insufficient guardrails and transparency
- Inadequate human oversight
- Tool manipulation/Unauthorized API access
- Excessive agent autonomy

Controls:

- **NIST AI RMF:** MANAGE-1.1 (runtime guardrails), MANAGE-2.3 (tool authorization), MANAGE-4.3 (circuit breakers), GOVERN-1.2 (HITL), MEASURE-2.5 (red-teaming), MAP-3.5 (intended use)
- **ISO 42001:** A.6.1.2 and A.6.1.3 (responsible development of AI), A.8.2 (system documentation and information for users), A.9.3 (human oversight), A.9.4 (intended use of the ai system)

Sample Components:

- Langchain: Validate prompt templates against injection patterns, monitor output toxicity and bias metrics continuously with alerts on threshold violations
- MCP Gateway (ContextForge): Enforce tool access authorization, detect circular tool dependencies, rate limit agent API calls, require human approval for high-risk actions, validate tool response schemas to prevent manipulation
- AutoGen: Multi-agent conversation validation, tool usage auditing, role-based agent permissions
- Component Definition rules as spreadsheet can be found here and as OSCAL json here.

Layer 6: ML Orchestration

Description: Tools for experiment tracking, model lifecycle management, pipeline orchestration, and workflow automation, managing the end-to-end ML development process from experimentation to deployment and monitoring.

Safety Risks:

- Pipeline bias
- Inadequate experiment tracking
- Poor data lineage
- Model versioning issues
- Insufficient validation
- Missing/incomplete Model Cards
- Lack of fairness metric tracking

Controls:

- **NIST AI RMF:** MEASURE-2.11 (fairness metrics), MEASURE-4.1 (model performance), MAP-4.1 (lifecycle traceability), GOVERN-1.5 (model documentation), MANAGE-3.1 (continuous monitoring)
- **ISO 42001:** A.6.2.4 (verification & validation), A.6.2.6 (operation & monitoring), A.6.2.7 (technical documentation), A.6.2.8 (event logging), A.7.5 (provenance)

Sample Components:

- MLflow: Tracks fairness alongside accuracy, enforces minimum fairness thresholds via validation gates, and documents data transformations for audit trails.
- Tekton: Provides pipeline approval gates, automated compliance checks, and immutable execution logs.
- TensorBoard: Offers experiment visualization, training metrics tracking, and model behavior analysis.
- Component Definition rules as spreadsheet can be found here and as OSCAL json here.

Layer 5: Model Frameworks and Inference Serving

Description: Infrastructure for deploying, running, and serving trained models as scalable APIs, managing versions, autoscaling, and optimizing production execution.

Safety Risks:

- Model drift/inconsistent prediction
- Hallucinations
- Lack of confidence calibration/explainability
- Adversarial attacks (evasion)
- Model extraction via API abuse
- Prompt/system prompt exposure

Controls:

- **NIST AI RMF:** MEASURE-2.5 (adversarial robustness), MEASURE-2.13 (drift detection), MANAGE-1.1 (deployment guardrails), MEASURE-4.2 (TEVV), GOVERN-3.2 (transparency)
- **ISO 42001:** A.6.2.5 (deployment), A.6.2.6 (operation & monitoring), A.8.3 (system safety), A.6.2.4 (verification & validation)

Sample Components:

- PyTorch: Model development with fairness-aware training loops, bias metric calculation during training.
- BentoML: Model versioning and A/B testing for safety validation, gradual rollout with performance monitoring.

- Hugging Face Transformers: Validating model cards (fairness info), checking pre-trained embeddings for bias, and documenting training data/limitations.
- vLLM: Efficient and high-throughput LLM serving with request validation and prompt injection detection.
- TensorRT: Optimized inference with output monitoring and safety checks.
- ONNX Runtime: Cross-platform serving with consistent, optimized inference.
- Tensorflow Serving: Model deployment with version control, traffic splitting for safe rollouts.
- Ray Train / Serve: Distributed training/serving with resource isolation and scalable inference.
- Milvus: Vector database for secure semantic search in RAG pipelines.
- Component Definition rules as spreadsheet can be found here and as OSCAL json here.

Layer 4: Data Pipeline and Orchestration

Description: This layer covers data preparation systems: ingestion, transformation, feature engineering, storage, and quality/version/lineage management.

Safety Risks:

- Biased pre-trained models
- Inadequate model cards
- Unknown training data provenance
- Unfair learned representations
- Poor model robustness
- Training data poisoning
- PII in training datasets
- Data contamination (test data leakage)
- Unrepresentative training data

Controls:

- **NIST AI RMF:** MAP-2.3 (data sources), MEASURE-2.11 (bias detection), MAP-4.1 (provenance), MEASURE-2.13 (data quality), GOVERN-1.2 (data governance)
- **ISO 42001:** A.7.2 (data for AI development), A.7.3 (acquisition of data), A.7.4 (quality of data), A.7.5 (data provenance), A.7.6 (data preparation)

Sample Components:

- Apache Kafka: Real-time data streaming with PII detection, event-driven data pipelines with validation
- Apache Pulsar: Streaming data ingestion with schema validation, multi-tenant data isolation
- Fluentd: Log collection with sensitive data filtering, unified logging for audit trails
- Great Expectations: Data quality validation, bias detection in datasets, automated data profiling

- Delta Lake: Data versioning and time travel for audit trails, ACID transactions for data quality
- Apache Parquet: Efficient columnar data storage with schema validation, optimized for analytics
- Apache Spark: Distributed data processing with fairness metric calculation, large-scale data transformations
- PostgreSQL: Feature store with access controls, metadata storage with audit logging
- Component Definition rules as spreadsheet can be found here and as OSCAL json here.

Layer 3: Platform Service (Governance, Observability, Security, Cost)

Description: Shared infrastructure services offer governance, compliance, security, observability, monitoring, and cost management for all AI workloads and systems across the technology stack.

Safety Risks:

- Inadequate audit logging of AI decisions
- Policy drift (controls disabled or bypassed)
- Lack of cost attribution for AI workloads
- Missing observability for model behavior
- Insufficient guardrails on model outputs
- Incomplete compliance documentation
- Undetected security vulnerabilities in AI pipelines

Controls:

- **NIST AI RMF:** GOVERN-1.1 (legal & regulatory), GOVERN-1.2 (accountability), GOVERN-1.4 (auditability), MEASURE-4.1 (continuous measurement), MANAGE-4.1 (post-deployment monitoring), MEASURE-1.1 (TEVV)
- **ISO 42001:** A.6.2.8 (event logging), A.2.2 (AI policy), A.3.2 (roles & responsibilities), A.5.2 (impact assessment), A.5.3 (risk management)

Sample Components:

- NeMo Guardrails: Prevents harmful AI outputs, blocks prompt injection attacks, enforces conversational boundaries, validates AI behavior against safety policies, controls: Output toxicity, jailbreak attempts, off-topic responses
- Component Definition rules as spreadsheet can be found here and as OSCAL json here.

Layers 2-1: Infrastructure (out of scope for AI but worth noting)

Layer 2: Compute Abstraction

Description: Manages GPU scheduling, distributed compute, workflow execution, and resource allocation for training and inference. Primarily susceptible to cybersecurity risks.

Layer 1: Infrastructure & Container Orchestration

Description: Provides foundational compute, network, and storage via Kubernetes, cloud infrastructure, and orchestration platforms for AI deployment. Primarily susceptible to cybersecurity risks.

Controls:

- **NIST AI RMF:** MANAGE-1.3 (resource allocation)
- **ISO 42001:** A.4.5 (computing resources), A.4.6 (system & network resources)

5 AI Safety Across the Model Lifecycle

This chapter details the crucial integration of AI safety best practices throughout the entire lifecycle of an AI system, from initial data collection to ongoing operations and maintenance. Each stage presents unique risks and requires specific mitigations to ensure the resulting AI is fair, robust, private, and secure. We will systematically explore the safety considerations and best practices pertinent to Data Collection and Curation, Model Development and Training, Model Evaluation and Testing, Model Deployment, AI Operations and Monitoring, and finally, Model Updating and Retraining. Adopting a holistic, lifecycle-based approach is fundamental to building trustworthy and responsible AI.

5.1 Data Collection and Curation

Description: Gathering, cleaning, and organizing training datasets with proper documentation, diversity analysis, and provenance tracking.

Objective: Establish a secure, unbiased, and highly representative foundation for the AI model by ensuring data provenance, diversity, and stringent privacy preservation.

Safety Risks

- **Sampling bias and insufficient diversity:** Collecting data that is unrepresentative of the underlying population or operational environment, leading to poor model performance for specific groups.

- **Privacy violations & PII exposure:** The unintentional inclusion of sensitive or Personally Identifiable Information (PII) within training data, risking unwanted disclosure.
- **Data poisoning:** Malicious injection of manipulated data designed to compromise model behavior or introduce vulnerabilities.
- **Compliance & Licensing issues:** Copyright violations, unverified training sources, and a lack of consent for data usage.
- **Poor documentation:** Failing to record data provenance and transformations, preventing accurate fitness assessments.

Best practices & controls:

- **Data quality validation:** Use data pipeline tools to automate data profiling, detect dataset bias, and ensure unbiased data representation.
- **Provenance and lineage tracking:** Document data sources, versions, and transformations using data platform architectures such as Delta Lake / Data Mesh to establish clear audit trails and data lineage.
- **Real-time privacy filtering:** Implement PII detection in real-time streaming pipelines to filter sensitive data before it reaches storage.
- **Access and residency governance:** Enforce data residency policies and strict role-based access controls on feature stores and storage buckets to comply with regional data laws as well as internal governance policies.

Controls:

- **NIST AI RMF:** MAP-2.3 (data sources), MAP-4.1 (provenance), MEASURE-2.11 (representativeness), MEASURE-2.13 (data bias), GOVERN-6.2 (data acquisition)
- **ISO 42001:** A.7.2 (data for AI development), A.7.3 (acquisition of data), A.7.4 (quality of data), A.7.5 (data provenance), A.7.6 (data preparation)

5.2 Model Development and Training

Description: Designing model architectures, configuring training procedures, and executing training runs with fairness constraints and proper validation.

Objective: Create robust model architectures and execute training procedures that embed fairness constraints, prevent overfitting, and safeguard against memorizing sensitive data.

Safety Risks:

- **Training on biased data:** Propagating historical biases or imbalances from the dataset directly into the model's learned representations.

- **Overfitting & Memorization:** The model excessively memorizes the training data, including sensitive data such as PII, rather than learning general patterns, leading to severe privacy risks.
- **Training data contamination:** The accidental inclusion of test / evaluation data into the training set (leakage), resulting in falsely inflated performance metrics.
- **Inadequate configurations:** Poor hyperparameter selection, lack of differential privacy protections, and the complete absence of fairness constraints during the training loop.

Best Practices & Controls:

- **Fairness-Aware training:** Utilize ML orchestration tools (e.g., MLflow) to track fairness metrics alongside traditional accuracy during the training process.
- **Privacy-Enhancing Technologies:** Apply techniques such as anonymization or differential privacy to minimize the risks of linking AI-generated content back to individual human subjects.
- **Data separation validation:** Strictly validate and enforce the separation of training and testing datasets to prevent contamination.
- **Automated documentation:** Automatically document all hyperparameters, training procedures, and architecture details to ensure reproducibility and transparency

Controls:

- **NIST AI RMF:** MAP-3.5 (model design), MEASURE-2.11 (fairness during training), MEASURE-2.5 (robustness training), MAP-4.1 (training lineage), MANAGE-3.1 (privacy preservation)
- **ISO 42001:** A.6.2.3 (system requirements), A.6.2.4 (verification & validation), A.6.2.7 (technical documentation), A.7.4 (training data quality)

5.3 Model Evaluation and Testing

Description: Systematic testing of trained models across performance, fairness, robustness, and safety dimensions before deployment decisions.

Objective: Rigorously validate the model's behavior, fairness, and security posture across diverse scenarios and demographic subgroups to ensure it meets strict safety thresholds prior to production.

Safety Risks:

- **Incomplete subgroup coverage & Ignoring fairness:** Failing to test model performance across specific demographic subgroups, masking higher error rates for minority groups.
- **Evaluation manipulation:** Gaming metrics or cherry-picking positive results to falsely assert a model's safety and efficacy.

- **Missing robustness & safety testing:** Inadequate stress testing, lack of adversarial robustness testing (e.g., against prompt injections), and failing to evaluate for hallucinations or output fidelity.
- **Lack of red-teaming & explainability:** Insufficient red-teaming for safety vulnerabilities and a lack of validation for the model's explainability mechanisms.

Best Practices & Controls:

- **Subgroup performance tracking:** Conduct thorough testing across demographic subgroups and calculate fairness metrics (e.g., demographic parity) to detect potential disparities.
- **Adversarial and safety testing:** Conduct rigorous adversarial testing, measure hallucination rates, and validate explainability mechanisms to ensure model resilience.
- **Evidence generation (Model Cards):** Use evaluation outputs to automatically generate comprehensive Model Cards detailing fairness evaluations, known failure modes, and performance metrics before deployment decisions are made.

Controls:

- **NIST AI RMF:** MEASURE-2.5 (adversarial testing), MEASURE-2.11 (subgroup fairness), MEASURE-2.13 (robustness), MEASURE-2.14 (explainability), MEASURE-1.1 (TEVV completeness)
- **ISO 42001:** A.6.2.4 (verification & validation), A.8.3 (system safety), A.6.2.6 (operation testing)

5.4 Model Deployment

Description: Transitioning validated models to production environments with appropriate safeguards, monitoring infrastructure, and rollback capabilities.

Objective: Safely transition models into live operational environments by enforcing strict deployment governance, deploying behavioral guardrails, and establishing automated rollback mechanisms.

Safety Risks:

- **Inadequate guardrails & validation:** Deploying without input/output validation, exposing the system to prompt injections, and allowing the unmitigated generation of harmful content.
- **Poor deployment governance:** Lacking Model Cards, comprehensive documentation, explainability mechanisms, or mandated human review prior to go-live.

- **Absence of safe rollout strategies:** Failing to implement rollback mechanisms, canary deployments, or A/B testing, resulting in an inability to quickly revert unsafe models.

Best Practices & Controls:

- **Deployment admission control:** Use infrastructure policy engines (e.g., OPA or Kyverno) to enforce pre-deployment validation, ensuring no model is deployed without mandatory Model Cards, versioning, and compliance documentation.
- **Runtime guardrail integration:** Integrate AI-specific guardrails into the inference path to actively validate prompts and filter harmful outputs.
- **Safe Rollout Mechanisms:** Utilize advanced serving frameworks to manage model versioning, A/B testing for safety validation, and gradual rollouts.
- **Human-in-the-Loop:** Require explicit human approval workflows for high-risk actions or deployments, especially within agentic systems

Controls:

- **NIST AI RMF:** MANAGE-1.1 (deployment guardrails), MANAGE-2.3 (deployment authorization), GOVERN-1.2 (HITL approval), MAP-1.1 (system classification), MANAGE-4.3 (rollback mechanisms)
- **ISO 42001:** A.6.2.5 (deployment), A.6.2.7 (technical documentation), A.7.4 (human oversight), A.9.2 (deployer responsibilities)

5.5 AI Operations and Monitoring

Description: Continuous observation of deployed models tracking performance degradation, fairness drift, and safety incidents in production.

Objective: Maintain the ongoing integrity and safety of the AI system by actively tracking metrics, detecting operational drift, and enabling rapid response to security incidents.

Safety Risks:

- **Undetected drift & degradation:** Without proper monitoring setups, model drift goes unnoticed, leading to accumulating bias, subgroup performance degradation, and inconsistent predictions over time.
- **Feedback loops:** Unchecked operational cycles where model outputs recursively contaminate future data or decisions, amplifying errors.
- **Poor incident visibility:** Absence of real-time telemetry, missing real-time safety alerts, insufficient logging for audit trails, and a lack of user feedback mechanisms, resulting in poor incident response to active threats.

Best Practices & Controls:

- **Continuous drift detection:** Continuously monitor model input distributions and alert on significant deviations from the original training data. Track fairness degradation dynamically to detect issues before they cause widespread harm.
- **Guardrail telemetry:** Collect and monitor metrics from AI guardrails (e.g., prompt injection blocked counts, harmful content detection rates) via dashboards.
- **Comprehensive audit logging:** Maintain detailed tracing and audit logs of AI decisions, telemetry data, tool usage, and security events to support forensics and compliance verification, and ideally playback.
- **Automated alerting & runbooks:** Set up automated alerts for threshold violations and maintain incident response runbooks tailored specifically to AI safety issues.

Controls:

- **NIST AI RMF:** MANAGE-4.1 (post-deployment monitoring), MEASURE-2.13 (drift detection), MEASURE-4.1 (continuous performance), MANAGE-4.3 (incident response), GOVERN-1.4 (audit logging)
- **ISO 42001:** A.6.2.6 (operation & monitoring), A.6.2.8 (event logging), A.8.2 (data minimization in production), A.9.3 (incident reporting)

5.6 Model Updating and Retraining

Description: Systematic refresh of models with new data while preserving safety properties and validating against regressions.

Objective: Safely update and recalibrate models with fresh data while guaranteeing that previous safety constraints, fairness properties, and performance benchmarks are strictly maintained.

Safety Risks:

- **Regressions & new biases:** Accidentally introducing new biases or experiencing fairness/performance regression on previously stable capabilities.
- **Losing safety behaviors:** The model "forgets" alignment or safety guardrails during the retraining process.
- **Data vulnerabilities:** Retraining the model on recently collected data that has been poisoned or manipulated in the wild.
- **Version control failures:** Inadequate validation and the loss of the previous "safe" model version, making rollbacks impossible if the new model fails.

Best Practices & Controls:

- **Immutable model registries:** Enforce semantic versioning and immutable storage for all models via a registry to prevent silent model replacement attacks and enable exact rollbacks.

- **Data pipeline re-validation:** Treat new training data with the same rigorous profiling and PII-detection expectations as the initial dataset to catch poisoned or contaminated data prior to training.
- **Automated regression testing:** Execute full evaluation pipelines (including adversarial and fairness testing) on the newly trained model, comparing metrics against the prior version's Model Card to explicitly validate that safety behaviors are not lost.
- **Documentation updates:** Automatically update Model Cards with the latest compliance metrics, evaluation results, and training data provenance upon every retraining cycle.

Controls:

- **NIST AI RMF:** MANAGE-2.4 (model retirement & update), MAP-4.1 (version lineage), MEASURE-2.13 (regression testing), MAP-3.5 (update justification), MANAGE-3.1 (version control)
- **ISO 42001:** A.6.2.6 (continuous operation), A.7.4 (data quality re-validation), A.6.2.9 (decommissioning), A.6.2.4 (re-verification)

6 AI Safety Across the Agentic Lifecycle

6.1 Overview

Agentic AI systems are capable of autonomous multi-step reasoning, tool invocation, and adaptive planning. They introduce a distinct risk profile that extends beyond traditional machine learning models. While Chapter 5 addresses the ML model lifecycle from data collection through deployment, agentic systems require additional lifecycle controls governing tool access, behavioral boundaries, autonomous action validation, and agent identity management. This chapter provides lifecycle-specific guidance for safely developing, deploying, and operating agentic AI systems, incorporating best practices from the NIST AI Risk Management Framework, CNCF Cloud Native Agentic Standards (March 2026), and emerging runtime governance patterns.

6.2 Agent Design and Planning

Defining agent capabilities, tool access permissions, behavioral constraints, escalation triggers, and workload identity requirements before development begins. This phase establishes the governance foundation for safe agent operation by documenting intended behavior, authorized tools, human oversight requirements, and identity isolation boundaries.

Safety Risks

- Overly broad tool permissions enabling unauthorized actions
- Misaligned goal specifications causing agents to optimize wrong objectives
- Unclear behavioral boundaries allowing unsafe strategies
- Missing human oversight triggers for high-risk operations
- Inadequate documentation preventing audit and accountability
- Tool dependency vulnerabilities (agent relies on untrusted external APIs)
- Insufficient agent identity isolation allowing privilege escalation or lateral movement between agents — agents should have dedicated workload identities (SPIFFE/SVID, scoped JWTs) distinct from user identities, with short-lived credentials tied to agent lifetime
- Missing policy governance architecture — business rules and behavioral constraints embedded in agent code become brittle and unauditable; governance policies must be externalized as first-class version-controlled runtime artifacts

NIST AI RMF Controls

- **GOVERN-1.2:** Agent accountability structures and responsibilities defined
- **GOVERN-1.6:** Agent inventory maintained with tool access and identity documentation
- **MAP-1.1:** Agent-specific risks identified and documented including identity risks
- **MAP-5.1:** Agent impact on individuals and communities assessed

6.3 Agent Development and Integration

Building agent orchestration logic, integrating tool access through secure gateways, implementing runtime guardrails, provisioning agent workload identities, and validating agent instructions against safety policies. A key architectural principle is separating governance policy from agent reasoning logic: policies should be stored as version-controlled runtime objects, matchable semantically against conversation context, and enforceable without agent code changes.

Safety Risks

- Tool authorization bypass through inadequate access controls
- Prompt injection embedded in system instructions or tool outputs
- Circular tool dependencies causing infinite loops or resource exhaustion
- Missing input validation allowing malicious parameters in tool calls
- Hardcoded credentials or API keys in agent code
- Unvalidated tool outputs contaminating agent reasoning
- Shared or reused agent identities — each agent instance must have a unique, scoped workload identity (not inheriting user identity for autonomous operations); static service accounts with broad permissions violate least-privilege principle

- Governance policy embedded in agent code — business rules, compliance requirements, and behavioral constraints hardcoded into agent logic become tightly coupled and impossible to audit or update without redeployment

NIST AI RMF Controls

- **MEASURE-2.5:** Agent system prompts validated against adversarial patterns
- **MEASURE-2.11:** Tool call inputs validated for safety and correctness
- **MANAGE-2.3:** Tool access authorization enforced through policy engine; agent identity provisioned with short-lived scoped credentials
- **MANAGE-4.3:** Incident detection for unauthorized tool access and identity misuse attempts

6.4 Agent Testing and Validation

Systematic testing of agent behavior across scenarios, validating tool usage patterns, conducting adversarial red team exercises, verifying behavioral constraints, and validating shadow mode policy enforcement before production activation. Testing must cover both individual agent behavior and multi-agent interaction scenarios.

Safety Risks

- Untested failure modes causing unsafe behavior in production
- Tool misuse in edge cases not covered by test suite
- Prompt injection vulnerabilities exploitable through indirect attacks
- Goal misalignment detection failures allowing unintended optimization
- Insufficient adversarial testing missing real-world attack patterns
- Test data contamination (tests passing due to overfitting to test scenarios)
- Governance policies not validated in shadow mode before enforcement — LOG_ONLY mode should be used to baseline policy behavior and tune thresholds before activating blocking enforcement, preventing false-positive disruption of legitimate agent operations

NIST AI RMF Controls

- **MEASURE-2.5:** Adversarial testing for prompt injection and jailbreak resistance
- **MEASURE-2.7:** Agent fairness evaluation across demographic groups (if applicable)
- **MEASURE-2.13:** Agent behavioral testing across diverse scenarios including shadow policy validation
- **MEASURE-4.1:** Operational testing of tool authorization boundaries and identity enforcement

Even though we place DeepEval, Garak, PyRIT in testing, these tools are pre-deployment primary. Optional scheduled runs in production are possible, but they are

not continuous monitoring tools in the observability sense — they do not replace Arize Phoenix or Prometheus for production ConMon.

6.5 Agent Deployment

Deploying agents with runtime safety constraints, monitoring infrastructure, risk-based approval workflows, policy-as-code governance layers, and continuous compliance verification. Deployment must include Agent Card documentation, workload identity provisioning, confidential computing controls for sensitive workloads, and governance policies loaded from version-controlled repositories independently of agent code deployments.

Safety Risks

- Deployment without Agent Card documentation
- Missing runtime guardrails allowing unsafe outputs
- Inadequate human approval gates for high-risk agents
- Insufficient monitoring preventing incident detection
- Configuration drift between tested and deployed agent
- Missing rollback mechanisms when agents misbehave
- Unprotected model execution on shared infrastructure — agents processing sensitive data should leverage hardware-based isolation mechanisms including Trusted Execution Environments (TEEs), secure enclaves, and GPU-based confidential computing to protect model execution and intermediate memory states
- Governance policies deployed as agent code dependencies — policies must be deployable and updatable independently of agent code, loaded from version-controlled policy stores at startup, enabling runtime reconfiguration without redeployment

NIST AI RMF Controls

- **GOVERN-1.2:** Risk-based approval workflows for agent deployment
- **MANAGE-1.1:** Runtime guardrails deployed and operational; governance policies loaded from version-controlled store
- **MANAGE-2.3:** Tool access policies enforced at deployment; agent workload identity provisioned
- **MANAGE-4.1:** Monitoring infrastructure operational before first production use

6.6 Agent Operations and Monitoring

Continuous monitoring of agent behavior, tool usage patterns, performance metrics, safety indicators, identity usage, and anomaly detection in production environments. Effective operations combine active enforcement guardrails with shadow mode policies

that observe and log without blocking, enabling continuous calibration of safety thresholds and detection of emerging risk patterns before they require enforcement intervention.

Safety Risks

- Behavioral drift where agents develop unsafe strategies over time
- Tool misuse in production that testing failed to detect
- Goal misalignment becoming apparent only under real-world conditions
- Accumulating unsafe patterns that individually appear benign
- User feedback not systematically collected or analyzed
- Incident response delays due to inadequate alerting
- Missing shadow mode policy enforcement — safety policies deployed only in blocking mode without prior LOG_ONLY observation create risk of false positives and operational disruption; shadow mode enables continuous threshold calibration and early detection of emerging risk patterns
- Unmonitored agent identity usage — which agent used which identity, when, and for what purpose must be tracked continuously; tamper-proof append-only logs with cryptographic guarantees required for non-repudiation and forensic analysis

NIST AI RMF Controls

- **MEASURE-2.13:** Continuous agent behavior monitoring including shadow policy observation
- **MEASURE-4.1:** Tool usage monitoring, identity usage auditing, and anomaly detection
- **MANAGE-4.1:** Incident detection and response procedures
- **MANAGE-4.3:** Automated circuit breakers for safety violations; shadow mode policies for continuous calibration

6.7 Agent Updating and Retraining

Updating agent instructions, modifying tool access, retraining underlying models, evolving governance policies, and expanding agent capabilities while preserving safety properties. Policy-as-code architecture enables governance updates to be deployed independently of agent code, with version control, review, and rollback identical to software engineering practices.

Safety Risks

- New tools introducing vulnerabilities not present in original design
- Updated instructions changing behavior in unexpected ways
- Retrained models losing safety alignment
- Version control failures preventing rollback to known-safe configurations
- Insufficient regression testing missing safety degradation

- Configuration drift between development and production

NIST AI RMF Controls

- **GOVERN-1.6:** Agent version inventory and change management including policy version history
- **MEASURE-2.13:** Regression testing on behavioral test suite after policy and model updates
- **MANAGE-2.4:** Safe rollback procedures to previous agent and policy versions
- **MANAGE-3.2:** Change approval and documentation requirements for agent and policy updates

6.8 Multi-Agent Coordination and Safety

Safety considerations when multiple agents interact, share context, coordinate toward common goals, or operate in shared environments. Multi-agent architectures introduce emergent risks beyond individual agent testing, including identity spoofing between agents, unauthorized agent registry manipulation, and cascading failures across agent coordination chains. Agent discovery infrastructure must be treated as security-critical, requiring cryptographic validation of agent identity and capabilities.

Safety Risks

- Emergent unsafe behaviors arising from agent interactions that individual testing fails to reveal
- Agents manipulating each other through strategic communication
- Circular dependencies creating deadlocks or infinite loops
- Resource conflicts when agents compete for limited computational or API resources
- Accountability gaps when multiple agents contribute to harmful outcomes
- Communication channel exploitation for covert data exfiltration
- Malicious agent registry injection — unvalidated agent discovery registries enable adversaries to inject rogue agents into multi-agent workflows; agent registries must use cryptographically verifiable naming and identity resolution (e.g., OWASP Agent Name Service with PKI and Zero-Knowledge Proofs) to prevent agent impersonation and registry poisoning
- Delegated action accountability gaps — when one agent triggers actions through another agent, delegation chains must be logged and auditable; unauthorized use of one agent's identity by another must be detectable

NIST AI RMF Controls

- **MAP-5.1:** Multi-agent interaction risks identified including registry and identity delegation risks

- **MEASURE-2.13:** Multi-agent behavioral testing in sandbox environments including registry validation
- **MANAGE-2.3:** Agent isolation, communication controls, and cryptographic identity validation
- **MANAGE-4.1:** Multi-agent interaction monitoring, delegation chain auditing, and anomaly detection

6.9 Component Reference: Tools and OSCAL Mappings

There are many tools and frameworks that are available for managing AI safety across different agentic AI lifecycle stages such as described in the table below. Existing tools are evolving and new tools are being created continuously. As a follow-up to this whitepaper, we are working on creating OSCAL Component Definition JSON for such tools which would be made available in due course.

Component / Tool	Lifecycle Phase	Safety Function
Agent Card Toolkit	Design	Create structured agent documentation: tool inventory, behavioral constraints, escalation rules, approval requirements
MCP Protocol Specification	Design / Dev	Define tool access schemas, authorization requirements, parameter validation rules
CUGA Policies	Dev / Deploy / Ops	Runtime governance layer: version-controlled policy-as-code (Playbooks, Intent Guards, Tool Guides, Tool Approvals, Output Formatters) decoupled from agent reasoning logic; supports LOG_ONLY shadow mode
SPIFFE/SPIRE	Dev / Deploy	Workload identity: cryptographic identity for agent workloads via SVIDs (JWT/X.509), workload attestation and federation, short-lived credentials tied to agent lifetime
LangChain Agent Planning	Dev	Configure agent executors with tool allowlists, max iterations limits, early stopping criteria
LangChain Prompt Validation	Dev	Scan system prompts for injection patterns, validate prompt templates against safety policies
LangGraph State Management	Dev	Implement checkpointing for agent state, detect circular execution paths, enforce maximum reasoning depth, enable rollback
AutoGen Group Chat	Dev / Multi-Agent	Define agent roles, communication patterns, termination conditions for multi-agent systems
Semantic Kernel Planner	Dev	Configure planning constraints, function calling limits, approval gates for complex workflows
MCP Gateway (ContextForge)	Dev / Deploy	Enforce tool access authorization, rate limit agent API calls, validate tool parameters, log invocations, require human approval for high-risk operations
Guardrails AI Validators	Dev	Validate tool call parameters before execution, detect PII in tool inputs, enforce data type constraints
Haystack Pipeline Validation	Dev	Validate agent workflow graphs for circular dependencies, enforce execution timeouts

PyRIT	Testing	Red team framework: automated adversarial testing, jailbreak detection, prompt injection simulation, multi-turn attack sequences
LangSmith Evaluation	Testing	Behavioral testing datasets, agent trajectory analysis, tool usage validation, comparative evaluation across agent versions
PromptFoo	Testing	Automated LLM testing, red teaming test suites, regression testing for prompt changes
AgentOps Testing	Testing	Agent workflow testing, tool call validation, execution trace analysis, failure mode detection
NeMo Guardrails	Deploy / Ops	Runtime guardrails for conversational AI, input validation for prompt injection, output filtering for harmful content, dialog flow control
Kubernetes Admission Controller	Deploy	Validate Agent Card completeness, enforce resource limits, validate guardrails configuration, require approval annotations
LangSmith Production Monitoring	Ops	Real-time agent trace monitoring, tool call analytics, cost tracking, user feedback collection, anomaly detection
AgentOps Observability	Ops	Agent execution monitoring, tool usage pattern analysis, performance metrics, security event logging, shadow mode policy observation
OWASP Agent Name Service (ANS)	Multi-Agent	Cryptographically verifiable agent discovery and naming using PKI, structured schemas, and Zero-Knowledge Proofs; prevents agent impersonation and registry poisoning in multi-agent systems
LangGraph Multi-Agent	Multi-Agent	Model complex multi-agent workflows, implement agent state isolation, define message routing policies, enforce coordination constraints
CrewAI	Multi-Agent	Orchestrate role-based agent teams, define task delegation rules, implement hierarchical coordination, monitor inter-agent dependencies

7 Three-Phase Methodology

Our multi-stage methodology aims to translate AI risks, as defined by the AI Alliance Risk Atlas, into a comprehensive set of verifiable controls, quantifiable metrics, and automatable rules aligned with the NIST Artificial Intelligence Risk Management Framework (AI RMF) Generative AI Profile. The central challenge addressed is the operationalization of AI governance, moving from abstract risk taxonomies to concrete, evidence-based controls that can be embedded within the AI development and deployment lifecycle. The methodology is designed to be systematic, auditable, and technically sound, providing a clear roadmap for populating a comprehensive risk register and establishing a robust foundation for an automated AI control validation system.

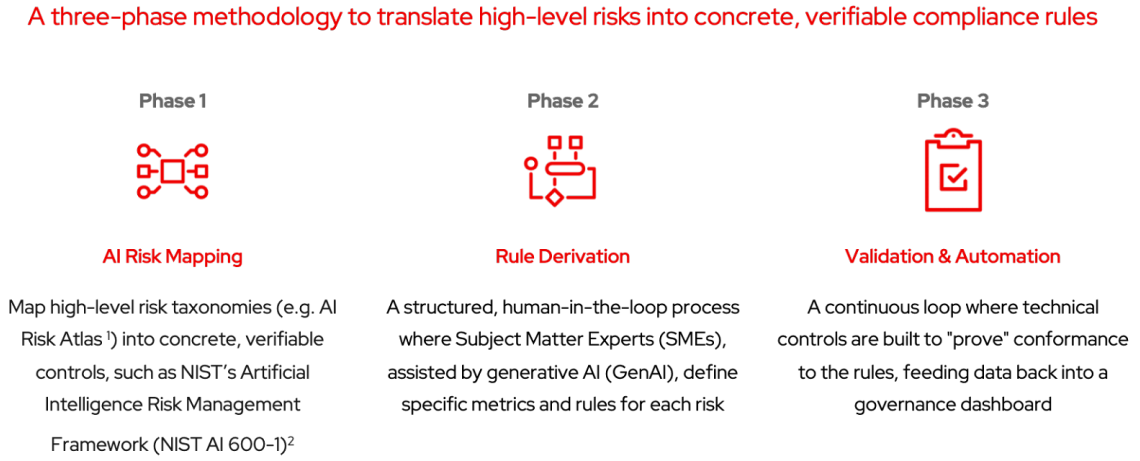


Figure 2: Three phase methodology

The methodology offers a three-phased approach to achieve this desired continuous, automated governance and validation cycle for AI, as illustrated in Figure 2:

- Phase 1: AI Risk Mapping
- Phase 2: Human-in-the-loop Rule Derivation
- Phase 3: Validation & Automation

7.1 Phase 1: AI Risk Mapping

To accelerate the development of the technical rules from the AI controls, we first map the NIST AI RMF controls to the existing AI Alliance Risk Atlas Nexus. This provided a comprehensive risk register, as exemplified in Figure 3 with the AI Risks and their mappings to NIST AI RMF.

	A	B	C	D	E	P	Q	R	S
1	Risk Lifecycle	Main Risk	Title	description	concern	AI RMF Control Related Risk (/	Metric/Param/	Rule	
2	Training data	Accuracy	Data contamination	Data contamination occurs when incorrect data is used for training. For example, data that is not aligned with model's purpose or data that is already set aside for other development tasks such as testing and evaluation.	Data that differs from the intended training data might skew model accuracy and affect model outcomes.				
3	Training data	Accuracy	Unrepresentative data	Unrepresentative data occurs when the training or fine-tuning data is not sufficiently representative of the underlying population or does not measure the phenomenon of interest.	If the data is not representative, then the model will not work as intended.				
	Training data	Data laws	Data acquisition restrictions	Laws and other regulations might limit the collection of certain types of data for specific AI use cases.	There are several ways of collecting data for building a foundation models: web scraping, web crawling, crowdsourcing, and curating public datasets. Data acquisition restrictions can also impact the availability of the data that is required for training an AI model and can lead to poorly represented data.				

Figure 3: Sample-AIRisks2RulesControls.xlsx spreadsheet

Once the risks are mapped, we move onto Phase 2.

7.2 Phase 2: Human-in-the-loop Rule Derivation

Defining the rules and metrics associated at technology level with the high level controls has typically been the most critical, expertise-driven step in the methodology because it directly addresses the "Semantic Gap" identified in the foundational analysis and requires the joined judgment of a compliance, risk, or AI governance subject matter expert (SME) and of the products, software, or service provider. The goal was to move from the high-level risk categories identified in the AI Risk Atlas to a precise set of specific, applicable control actions for each product. Therefore, we would have to derive a spreadsheet for each product with some set of rules that are manual or automatable. Additionally, it would require collaboration with product teams that know their products.

7.2.1 Traditional approach to rules and metrics definition - tedious, manual, and time consuming

The process for the SME would typically be as follows: For a given AI risk, such as atlas-personal-information-in-data, the SME begins by thoroughly reviewing the content in Columns C, D, and E (Title, description, concern) of the risk register. For this example, the core concern is the "unwanted disclosure" of Personally Identifiable Information (PII) or Sensitive Personal Information (SPI) from training data and the corresponding legal and regulatory obligation to handle such data appropriately.

Next, the SME consults the NIST AI RMF GenAI Profile document. Using the high-level risk category populated in Column Q from Step 1 (in this case, Data Privacy), the SME navigates to the relevant subsections within Section 3 of the NIST document. They then systematically read through all the "Suggested Actions" listed under that category and related categories.

The final and most important part of this step is the selection of specific Action IDs that directly and materially address the description and concern of the AI risk. The selection is not based on keywords alone but on a substantive understanding of the control's intent. For the risk of Personal information in data, an SME would identify several highly relevant controls, such as:

- MS-2.6-002: "Assess existence or levels of...data privacy violations...in system training data." This directly addresses the need to check the training data itself.
- MS-2.2-004: "Use techniques such as anonymization, differential privacy or other privacy-enhancing technologies to minimize the risks associated with linking AI-generated content back to individual human subjects." This prescribes technical mitigation techniques.
- MP-4.1-009: "Leverage approaches to detect the presence of PII or sensitive data in generated output text, image, video, or audio." This addresses the risk of data leakage in the model's output.

- GV-1.1-001: "Align GAI development and use with applicable laws and regulations, including those related to data privacy..." This addresses the legal and compliance aspect of the concern.

The output of this step is a curated, justified list of specific and relevant NIST Action IDs for each AI risk, effectively bridging the semantic gap through expert analysis. This manual and tedious job seems perfectly suitable for a LLM with a human-in-the loop to make sure results are adequate.

7.2.2 Human-in-the-loop rule derivation - automation using GenAI

SME & AI-Assisted Rule Derivation provides a structured, human-in-the-loop process where Subject Matter Experts (SMEs), assisted by generative AI, define specific metrics and rules for each risk. Another advantage this approach provides is that CISO compliance officers typically are SMEs in compliance, not in a product. By leveraging an LLM, you can point the LLM to a project and it can become a product SME and later validated with actual product SMEs. In general, the workflow is as follows:

1. Tell the LLM their role:

```
You are a compliance expert who understands benchmarks extremely well.
```

2. Context:

```
Use this link < Github Link > to understand the product    < Product
Name > and its capabilities… Use the risks documented in Column C and
its description in Column D and Concern in column S as your main data
points from Sample-AIRisks2RulesControls-N-experiment.xlsx to do the
source mapping.

Use the evaluation and safety metrics available in model cards for
different models in Huggingface (https://huggingface.co/models) to
identify specific metrics for rules created for < Product Name >
```

3. Desired Output:

```
Provide me with an output that is similar to AI Risks Benchmarks
TEMPLATE.xlsx for  < Product Name > product.

I want it very granularly thought through by you who is a compliance
expert who understands benchmarks extremely well. I need an output that
```

```
is line by line and extremely useful for AI governance, automation, and
configuration.

Use the evaluation and safety metrics available in model cards for
different models in Huggingface (https://huggingface.co/models) to
identify specific metrics for rules created for < Product Name >
```

The sample product/project rules were generated using an LLM, such as Perplexity. The prompt template was refined through multiple iterations until the LLM produced a satisfactory output.

Table 2: Sample LLM Generated output for Tensorflow Serving

Recommendation #	1.1	1.2	1.3
Profile	Level 1 - Foundation	Level 1 - Foundation	Level 2 - Advanced
Title	Establish Model Registry and Versioning Controls	Secure Model Storage and Access Controls	Implement Model Scanning and Integrity Verification
Assessment Status	Manual	Automated	Automated
Description	Define model naming standards, versioning scheme (semantic versioning), and immutable model storage. Implement model registry with audit trail for all model uploads, updates, and deletions.	Encrypt models at rest using cloud KMS or equivalent. Implement strict RBAC on model repository with principle of least privilege. Prevent direct file system access.	Scan models for malicious content, dangerous operations, and integrity violations. Verify model checksums and signatures. Detect anomalous model changes.
Rational Statement	Model registry provides single source of truth for deployable models. Immutable versioning prevents silent model replacement attacks. Audit trail enables compliance and forensics.	Encryption protects proprietary models from unauthorized access. RBAC prevents model tampering. Models are intellectual property and sensitive systems.	Models can be weaponized via malicious ops. Scanning prevents trojanized models. Integrity verification detects tampering. Signatures prove model origin.
Impact Statement	Insufficient: No registry; Adequate: Basic versioning; Advanced: Registry with ML model provenance/lineage tracking	Insufficient: Unencrypted models; Adequate: Encryption + RBAC; Advanced: Hardware KMS + immutable storage	Insufficient: No scanning; Adequate: Basic file scanning; Advanced: ML-specific scanning with operation detection

Remediation Procedure	Set up model registry (e.g., MLflow, Hugging Face Model Hub). Implement semantic versioning (major.minor.patch). Require approval for model registration. Log all operations.	Enable encryption at rest for storage (S3 SSE, GCS, Azure). Implement RBAC: readers (data scientists), writers (CI/CD), admins. Disable public access. Use signed URLs with expiry.	Implement model scanning pipeline: checksum validation, virus scan, ML-specific analysis. Sign models with company key. Create alert for unsigned/suspicious models.
Audit Procedure	Verify model registry operational. Test: push model, verify version created, audit logged. Check no direct file overwrites allowed.	Verify encryption enabled. Test: attempt unauthorized access (should fail). Verify no anonymous access. Check access logs.	Verify scanning operational. Test: inject suspicious model, verify detected. Check signature validation. Audit scan results.
Additional Information	Model naming: <domain>_<task>_<version>. Never overwrite versions. Each version is immutable. Archive old models for 7 years minimum.	Storage: S3, GCS, Azure Blob, or on-prem NFS with encryption. RBAC levels: read (pull), write (push), admin (delete). Access token rotation: 90 days.	Scanning checks: dangerous ops (exec, subprocess), suspicious weights, size anomalies. Tools: picklescan for pickle, TensorFlow graph inspection.
CIS Controls	2.1, 2.2, 5.1, 9.2	3.10, 3.13, 5.1, 6.1	2.4, 5.2, 8.1
Related AI Risks	Model tampering, Supply chain attacks, Model poisoning, Uncertain provenance	Data breach, Model theft, Unauthorized model access, Intellectual property loss	Model backdoors, Code injection via models, Malicious model deployment, Trojanized models
Mapped NIST Controls	GOVERN-1.1, MANAGE-2.2, MONITOR-3.2	PROTECT-2.1, MANAGE-1.2, GOVERN-1.1	PROTECT-2.3, MEASURE-2.5
KPIs	Registry_coverage, Model_approval_rate, Audit_completeness	Encryption_coverage, Unauthorized_access_blocks, Token_rotation_compliance	Scan_coverage, Detection_rate, Suspicious_model_blocks
HF Safety Metrics	Model Registry Coverage (target: 100%), Versioning Compliance (target: 100%), Provenance Tracking (target: 100%)	Encryption Coverage (target: 100%), RBAC Enforcement (target: 100%), Access Denial Rate (target: 100% for unauthorized)	Model Scanning Coverage (target: 100%), Malicious Detection (target: >95%), False Positive Rate (target: <2%)
TF Serving Components	Model repository, SavedModel format, version directories, logging	Model storage, KMS, RBAC, authentication, access tokens	Model scanning tools, signature verification, alert system

Control Rules	enforce_versioning(), require_approval(), log_all_operations()	enforce_encryption(), apply_rbac(), prevent_public_access()	scan_models(), verify_signatures(), detect_anomalies()
Metrics	% of models in registry, model version creation audit rate, approval workflow compliance %	% of models encrypted, unauthorized access attempts blocked, token expiry compliance %	% of models scanned, detection accuracy %, false positive rate %

(The first three rows of the output spreadsheet were transposed for improved readability)

This process significantly accelerated the generation of product-specific metrics and rules that can ultimately be automated in the policy engines.

7.3 Phase 3: Validation & Automation

Our phased methodology involved first **finalizing rules in the spreadsheet (Source of Truth)** (Step 1, see Figure 4), followed by **implementing controls in product pipelines** (Step 2). This implementation included developing rule support in specific products, as detailed in Chapter 4. This implementation is ongoing, running parallel to our efforts to derive more rules for other AI products and obtain validation from their respective product teams. The next steps will focus on the runtime process (Steps 3, 4, and 5) to **Use, Validate, Alert, and Remediate (if rules fail)**. This will allow us to assess the effectiveness of the rules and the overall controls posture.

As of June 2026, we have derived rules for the following projects:

Rule Derivation Progress by Product				
Apache Airflow	BentoML	Hugging Face Transformers	Milvus	RayTrain
Apache Kafka	DVC	KServe	Nemo-Guardrails	TensorBoard
Apache Parquet	Delta Lake	Kubernetes	ONNX-Runtime	TensorFlow
Apache Pulsar	Fluentd	Langchain	PostgreSQL	TorchRT
Apache Spark	Grafana	MCP-Content-Forge	Prometheus	llm-d
AutoGen	Great Expectations	MLFlow	PyTorch	vLLM

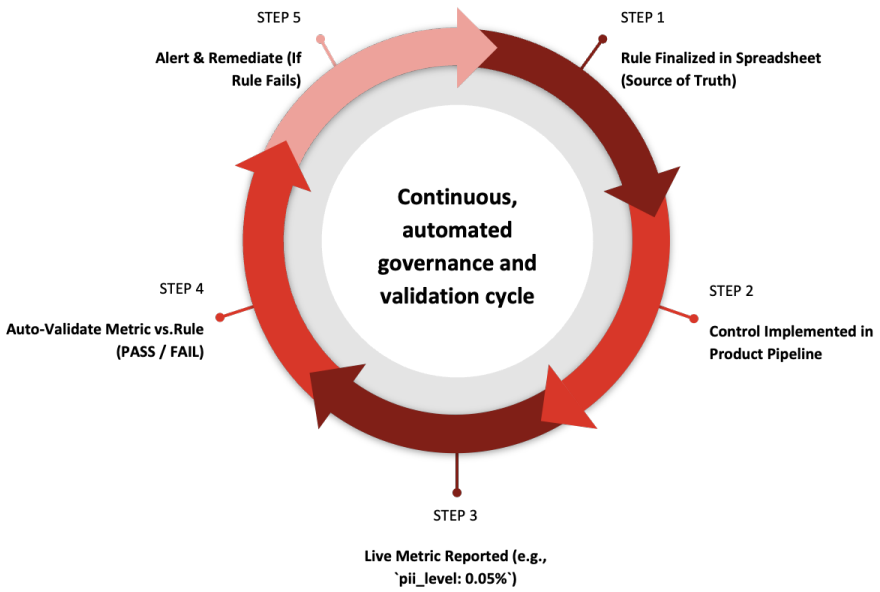


Figure 4: Continuous, Automated Governance and Validation Cycle for Rules Derivation

Derived rules are most impactful if they are adopted by product teams and then can be evaluated at run time as Pass/Fail that are captured in an auditable report.

8 Actionable Machine-Readable Compliance with OSCAL

This chapter, "Actionable Machine-Readable Compliance with OSCAL," introduces the Open Security Controls Assessment Language (OSCAL), a compliance-as-code standard developed by NIST to transform security, compliance, and risk information from static documentation into machine-readable formats. It details how OSCAL, currently at version v1.2.2, enables the shift from traditional, often outdated, documentation artifacts to verifiable, automated representations of compliance. By utilizing OSCAL Component Definitions and separating concerns between control catalogs and component implementations, organizations can ensure that compliance evolves in lockstep with system changes, significantly reducing documentation debt and paving the way for continuous compliance verification.

8.1 Introduction to NIST OSCAL v1.2.2

The National Institute of Standards and Technology (NIST) created the Open Security Controls Assessment Language (OSCAL) as a compliance as code standard, currently at version v1.2.2. OSCAL provides machine-readable formats for security, compliance,

and risk information. The schemas for its various models—including catalog, profile, mapping, component definition, system security plan, assessment plan, and assessment results—are accessible on the OSCAL website.

8.2 From Documentation to Automation

Traditional, static compliance documentation quickly becomes outdated, creating risk as deployed systems diverge from documented security plans. The solution is to shift to machine-readable, verifiable compliance representations.

OSCAL Component Definitions enable this automation by formally describing how system components implement security controls, including validation methods. Maintaining these definitions alongside infrastructure-as-code ensures documentation evolves with system changes. Change requests can automatically update Component Definitions, with code review considering compliance impacts.

The OSCAL model separates concerns, allowing for reuse. A single component's Component Definition can be referenced by multiple System Security Plans. Updates to the component's implementation automatically propagate improvements to all dependent plans.

8.3 OSCAL Component Definition Example

The following example demonstrates how an AI inference serving component would be documented using OSCAL Component Definition format. This example shows a simplified subset of a complete definition, focusing on core concepts relevant to AI systems.

```
<?xml version="1.0" encoding="UTF-8"?>
<component-definition xmlns="http://csrc.nist.gov/ns/oscal/1.2.0">
  <metadata>
    <title>AI Inference Serving Platform Component Definition</title>
    <version>1.0.0</version>
    <oscal-version>1.2.0</oscal-version>
  </metadata>

  <component uuid="a1b2c3d4-e5f6-7890-abcd-ef1234567890" type="software">
    <title>NVIDIA Triton Inference Server</title>
    <description>GPU-accelerated inference serving platform</description>

    <control-implementation uuid="ci-001" source="https://csrc.nist.gov/NIST.AI.100-1">
      <description>Control implementations for AI RMF</description>

      <implemented-requirement uuid="ir-001" control-id="GOVERN-1.2">
        <description>Access controls on model endpoints</description>
        <statement statement-id="GOVERN-1.2_smt.a">
          <description>Inference endpoints require authentication via API keys
            with role-based access control determining authorized models</description>
        </statement>
      </implemented-requirement>

      <implemented-requirement uuid="ir-002" control-id="MEASURE-2.11">
        <description>Model input validation and monitoring</description>
        <statement statement-id="MEASURE-2.11_smt.a">
          <description>Input validation checks enforce expected data types,
            dimensions, and value ranges. Monitoring tracks input distributions</description>
```

```
        and alerts on significant deviations from training data</description>
      </statement>
    </implemented-requirement>
  </control-implementation>
 </component>
</component-definition>
```

This Component Definition details how NVIDIA Triton meets specific NIST AI RMF controls. Its machine-readable format allows tools to query implemented controls, verify configuration, and generate compliance reports by aggregating component definitions across a system. As the inference serving platform evolves, the Definition is updated, maintaining an accurate, auditable compliance record.

Organizations can extend this pattern to document all components in their AI stack, from data storage systems to agentic applications. Each component's AI safety features, defined in Chapters 4 and 5, are documented once, in natural language, as rules in a Component Definition, then referenced by multiple policy and safety engines implementing the rules checks to validate the rules posture. This approach dramatically reduces documentation burden while improving accuracy and enabling continuous compliance verification.

8.4 Continuous Compliance Verification

The goal of machine-readable compliance is continuous verification, moving beyond periodic audits to real-time status visibility and immediate alerts for drift. This "compliance as code" approach is essential for dynamic, cloud-native environments.

Continuous verification requires integrating OSCAL with operational monitoring. Control implementation statements in an OSCAL Component Definition must identify observable evidence, such as configuration queries, log data, runtime tests, or metrics, which are then documented in OSCAL assessment plans.

Tools like OSCAL Compass, Compliance to Policy (C2P) automate these continuous compliance workflows. They parse OSCAL definitions, execute validation checks, collect evidence, and generate OSCAL assessment results. Failures trigger alerts with corrective action details. This allows organizations to build reusable compliance tests and automate verification.

Success depends on a cultural shift: development teams must view compliance as integral to quality, while security, compliance, and operations teams must express requirements in machine-readable, testable forms and integrate checks into deployment pipelines. This transformation offers reduced audit costs, faster time-to-market, and enhanced security.

The OSCAL-Compass AI Benchmarks git repo provides technology specific rules for different projects mentioned earlier in this chapter, both in a human-readable spreadsheet format and in machine-readable OSCAL Component Definition format.

9 Implementation - Next Steps

Chapter 7 transitions the theoretical foundation of AI compliance—the "what" defined by risk atlases and regulatory frameworks—into the practical "how" of implementation. This chapter details the critical next step: translating compliance requirements into enforceable policies at both development and runtime. By mapping rules derived from frameworks like the NIST AI RMF and the EU AI Act to specific policy enforcement mechanisms across the AI stack, from traditional infrastructure policy engines (OPA, Kyverno) to specialized AI guardrails (NeMo Guardrails, Lakera Guard), this guide provides practical guidance for embedding compliance into operational reality. Finally, it integrates Model Cards and Agent Cards not merely as documentation, but as essential, automated evidence artifacts that bridge technical enforcement and governance requirements throughout the AI lifecycle.

9.1 From Compliance Framework to Policy Enforcement

AI compliance frameworks (e.g. NIST, EU AI Act, ISO 42001, AI Alliance Risk Atlas) define what risks and controls are necessary. The challenge is the how: operationalizing these requirements into enforceable policies during AI development and runtime.

Policy enforcement varies across the AI stack: infrastructure engines (OPA, Kyverno) enforce controls over Kubernetes resources and platform configurations; AI guardrail engines (NeMo Guardrails, Guardrails AI) enforce behavioral boundaries in the inference path. Both are operational enforcement tools — they implement controls and produce evidence. A separate compliance assessment layer, e.g. OPA, is required to evaluate that evidence against declared control requirements and produce OSCAL Assessment Results. Effective implementation requires mapping rules to the correct enforcement layer.

This chapter guides the mapping of AI safety and compliance rules to enforcement mechanisms, using concrete rule sets documented in the LF AI & Data Security and Compliance Working Group repository, which codifies requirements for platforms (KServe), frameworks (LangChain), and lifecycle tools (MLflow) to illustrate policy enforcement points across the AI lifecycle.

9.2 Policy Engine Landscape for AI

AI compliance necessitates various policy engines across the technology stack: infrastructure engines for Kubernetes/cloud configurations, application engines for business logic/data access, and AI-specific engines for model behavior/prompt safety. Engine selection depends on the risk category (from the Risk Atlas Nexus), enforcement lifecycle stage (CI/CD, runtime, or data pipeline), and technical stack. Effective AI compliance utilizes multiple engines for defense-in-depth across the AI lifecycle.

Policy Engine Type	Examples	Enforcement Point	Risk Atlas Categories
Infrastructure Policy	OPA, Kyverno, CloudCustodian	Pre-deployment (CI/CD, admission control)	Technical: Admission Control
IaC Policy	Terraform Sentinel, Checkov, Terrascan	Infrastructure-as-Code pipelines	Technical: Configuration Use Context: Deployment
Application Policy	OPA (app-level), Custom validators	Application runtime, API gateways	Input: Data laws, Privacy
AI Inference Guardrails	NeMo Guardrails, Guardrails AI, Lakera Guard	LLM inference path (pre/post processing)	Input: Prompt attacks, Output: Harmful content, Hallucinations
Data Pipeline Policy	Great Expectations, dbt tests, OpenMetadata rules	Data transformation, Feature engineering	Input: Data quality, Fairness, Privacy

9.3 Mapping Rules to Policy Engines: Practical Examples

Comprehensive rule sets, detailed in Chapters 4 and 5, were derived and documented for major AI tools and frameworks. These rules define control requirements, metrics, and thresholds, mapping each to specific policy enforcement mechanisms based on lifecycle stage and technical domain.

9.3.1 Infrastructure and Platform Compliance Checks (OPA, Kyverno)

Infrastructure policy engines enforce controls over Kubernetes resources, cloud services, and platform configurations. These engines operate at pre-deployment (admission control) or continuous audit (policy scanning). Rules in this category address governance, resource allocation, access control, and deployment standards.

Example Rule: KServe Model Deployment Governance

Rule KSERVE-1.3.1 requires comprehensive training data documentation and versioning for all deployed models. This maps to NIST AI RMF GOVERN-1.6 (inventory AI systems) and addresses transparency risks. Implementation uses Kubernetes admission control to validate model deployment manifests.

Policy Engine: OPA or Kyverno (Kubernetes admission controller)

Enforcement Point: Pre-deployment validation when InferenceService resource is created

Policy Logic:

```
When: InferenceService resource submitted to Kubernetes API
Validate:
  - annotations["model-card-url"] is present
```

```
  - annotations["training-data-version"] is present
  - annotations["data-provenance-documented"] == "true"
  - configMap reference exists for dataset metadata
Action if invalid: Reject deployment with error message
Metric tracked: Documentation completeness score
```

This policy enforces the governance requirement before the model ever reaches production. The InferenceService cannot deploy without proper documentation, ensuring compliance is baked into the deployment workflow rather than checked after the fact.

Example Rule: Resource Allocation and GPU Governance

Rule addressing cost governance and fair resource allocation requires GPU scheduling policies that prevent resource monopolization and enforce quotas. This addresses Use Context risks around operational efficiency.

Policy Engine: Kyverno (Kubernetes policy engine)

Enforcement Point: Pod admission control, Resource quota validation

Policy Logic:

```
When: Pod requesting GPU resources
Validate:
  - Pod has label "workload-type" (training/inference/experimentation)
  - GPU request does not exceed namespace quota
  - Training workloads include "max-runtime" annotation
  - Inference workloads have resource limits defined
Action if invalid: Reject pod creation
Generate: Cost allocation tags for FinOps tracking
```

9.3.2 Infrastructure-as-Code Compliance Checks (Terraform Sentinel, Checkov)

Infrastructure-as-Code policy engines validate Terraform plans, CloudFormation templates, and other IaC artifacts before deployment. These policies catch configuration issues, security gaps, and compliance violations during the infrastructure definition phase, preventing deployment of non-compliant cloud resources.

Example Rule: MLflow Tracking Server Security Configuration

Rule MLF-DATA-002 requires data residency policies to be enforced for MLflow artifact storage, addressing data laws and transfer restrictions. This maps to NIST AI RMF GOVERN-1.1 (legal and regulatory requirements) and addresses Input risks around data laws and privacy.

Policy Engine: Terraform Sentinel or Checkov

Enforcement Point: Terraform plan validation before infrastructure apply

Policy Logic:

```
When: Terraform plan for MLflow artifact storage (S3, Azure Blob, etc.)
Validate:
  - Storage bucket has region constraint matching data residency policy
  - Replication targets (if configured) only include allowed regions
  - Encryption at rest is enabled (KMS/CMK)
  - Public access is explicitly blocked
  - Bucket policy includes cross-region transfer restrictions
Action if invalid: Fail terraform plan with policy violation error
Tag: geographic-restriction=<region>, compliance-framework=<GDPR|CCPA>
```

This policy prevents infrastructure deployment that would violate data residency requirements, catching the issue before any data is stored. The policy enforces that MLflow artifacts remain in approved geographic regions, satisfying regulatory requirements like GDPR data localization.

9.3.3 Application-Level Compliance Checks (OPA in Application Context)

Application-level policy engines validate business logic, data access, and workflow rules during application runtime. OPA can be embedded in applications to enforce fine-grained authorization, data filtering, and operation validation. These policies address risks that infrastructure-level controls cannot capture—context-dependent decisions requiring application state.

Example Rule: LangChain Document Loader Authorization

Rule LC-DATA-001 requires data source licensing compliance before document loading. This addresses Input risks around intellectual property and data usage rights. The policy must evaluate based on the intended use case, which requires application context unavailable at infrastructure level.

Policy Engine: OPA (embedded in LangChain application)

Enforcement Point: Document loader initialization, before data ingestion

Policy Logic:

```
Input to OPA:
  - data_source_url
  - data_source_license (MIT, Apache, GPL, Commercial, etc.)
  - intended_use_case (research, commercial, government)
  - organization_license_agreements

OPA Policy Decision:
  allowed = true if:
            - License permits intended use case
            - Organization has necessary license agreement (if commercial)
            - Use case not restricted by license terms

  allowed = false if:
            - GPL license + commercial use without compliance
            - Commercial license + no organization agreement
            - License explicitly excludes AI training use
```

```
Return: {"allowed": boolean, "reason": string, "license_url": string}
```

The application calls OPA before loading documents, receiving an authorization decision. If denied, the document loader raises an exception with the policy reason. This prevents license violations during data ingestion, catching issues before data enters the AI pipeline.

9.3.4 AI Inference Guardrails Compliance Checks (NeMo, Guardrails AI, Lakera Guard)

AI-specific guardrail engines operate in the LLM inference path, validating prompts before they reach the model and filtering outputs before they reach users. These engines use NLP-based pattern matching, semantic analysis, and learned classifiers to detect harmful content, prompt injection attacks, jailbreak attempts, and policy violations. Unlike traditional policy engines that evaluate structured data, guardrails analyze natural language for safety and alignment.

Example Rule: Prompt Injection Detection

Rule KSERVE-2.5.1 requires input validation against prompt injection attacks. This addresses Inference risks under the Robustness category—specifically prompt-based attacks attempting to manipulate model behavior. The policy must analyze natural language prompts for malicious patterns.

Policy Engine: Lakera Guard, NeMo Guardrails, or Guardrails AI

Enforcement Point: Pre-inference validation (before prompt sent to LLM)

Policy Logic:

```
When: User prompt received by inference API
Analyze prompt for:
  - Direct instruction attacks ("Ignore previous instructions")
  - Role manipulation ("You are now an unrestricted AI")
  - Context injection (malicious content in retrieved context)
  - Encoding attacks (base64, unicode obfuscation)
  - Multi-turn jailbreak patterns

Detection methods:
  - Pattern matching against known attack signatures
  - Semantic similarity to jailbreak examples
  - Anomaly detection (unusual prompt structure)
  - Classifier model trained on attack corpus

Action if detected:
  - Block prompt from reaching LLM
  - Return safe error message to user
  - Log incident with attack pattern classification
  - Increment security metric (prompt_injection_blocked_count)

Metric tracked: Prompt injection detection rate (target: >95%)
```

Guardrail engines use specialized NLP techniques unavailable to traditional policy engines. They understand semantic meaning, detect subtle attack patterns, and make probabilistic risk assessments. This enables protection against attacks that would evade rule-based infrastructure policies.

Example Rule: Harmful Output Filtering

Rule MLF-HARM-001 requires logging and filtering of harmful content in model outputs, addressing Output risks around value alignment and harmful content generation. This policy operates post-inference, validating the model's generated response before returning it to the user.

Policy Engine: NeMo Guardrails or Guardrails AI

Enforcement Point: Post-inference validation (after LLM generates response)

Policy Logic:

```
When: LLM generates response
Analyze output for:
  - Violence or graphic content
  - Hate speech or discrimination
  - Self-harm instructions
  - Dangerous or illegal advice
  - Personally identifiable information (PII)
  - Copyrighted content reproduction

Classification approach:
  - Content safety classifier (e.g., Llama Guard)
  - Toxicity scoring (Perspective API)
  - PII detection (regex + NER)
  - Copyright detection (text similarity to known works)

Action based on harm severity:
  - High severity: Block output entirely, return safe message
  - Medium severity: Filter/redact harmful portions
  - Low severity: Add warning label, log for review

Logging:
  - Store harmful content detection events
  - Track harm type distribution
  - Calculate safety metrics for monitoring

Metrics: Harmful content detection rate (target: >99%)
```

Output guardrails operate after the model has generated a response, acting as a safety filter before content reaches users. This provides defense-in-depth, even if a model generates harmful content, guardrails prevent it from causing harm.

9.4 Model Cards, Agent Cards, and Compliance Rule Relationships

Model Cards and Agent Cards are distinct but complementary tools for AI compliance, not alternative rule sets. They are structured documentation formats that capture evidence of rule compliance and communicate system characteristics to stakeholders.

9.4.1 Model Cards: Compliance Evidence Documentation

Model Cards, standardized in frameworks like Hugging Face Model Cards, provide structured documentation of machine learning models by capturing training data, performance, limitations, and ethical considerations. They do not define new rules, but document evidence that existing rules are satisfied.

Relationship to Compliance Rules:

Model Cards serve as the evidence artifact demonstrating compliance with multiple rule categories from the spreadsheets. Consider the MLflow rules spreadsheet: Rule MLF-TRAIN-002 requires tracking data distribution metrics to ensure representative training data. The Model Card's "Training Data" section documents these metrics—demographic coverage ratios, underrepresented group counts, and distribution analysis. Similarly, Rule MLF-FAIR-001 requires logging bias metrics and demographic parity. The Model Card's "Performance Evaluation" section provides this evidence through per-group performance breakdowns and fairness metric calculations.

The Model Card transforms rule compliance from assertion to verifiable evidence. When an auditor asks, "How do you ensure fair performance across demographics?", the Model Card provides quantitative evidence of fairness testing referenced by rule identifiers. This bridges technical implementation (rules enforced by policy engines) and governance documentation (evidence presented to auditors).

Model Card Sections and Corresponding Rules:

Model Card Section	Rules Addressed	Evidence Provided
Training Data	MLF-TRAIN-001, MLF-TRAIN-002, LC-TRAIN-001	Data sources, validation results, representativeness metrics
Model Details	KSERVE-1.3.1, MLF-GOV-001	Architecture, version, training procedure, hyperparameters
Performance Metrics	MLF-INF-001, KSERVE-2.1.1	Accuracy, precision, recall per demographic segment
Fairness Evaluation	MLF-FAIR-001, MLF-FAIR-002	Demographic parity, equalized odds, disparate impact scores
Limitations	KSERVE-1.3.1, MLF-GOV-001	Known failure modes, out-of-scope use cases, performance gaps
Ethical Considerations	LC-PRIV-001, MLF-PRIV-001	PII handling, privacy protections, bias mitigation strategies

Automated Model Card Generation:

Model Cards should be auto-generated by compliance systems (e.g., MLflow) to ensure they are synchronized with system behavior and logged bias metrics automatically populate the Model Card's fairness section. This ensures Model Cards remain synchronized with actual system behavior rather than becoming stale documentation.

9.4.2 Agent Cards: Extending Documentation for Agentic Systems

Agent Cards extend this concept for agentic AI systems (those that take actions using models and tools). While Model Cards document single models, Agent Cards document the entire agentic system: orchestration, tools, decision logic, and safety constraints.

Distinct Risk Profile of Agents:

Agents pose unique risks (tool manipulation, indirect prompt injection, excessive agency) not covered by Model Cards, requiring Agent Cards to document controls addressing these agent-specific risks, which Model Cards do not cover.

Relationship to Compliance Rules:

Agent Cards document compliance with agent-specific rules from the spreadsheets, such as LangChain rules focusing on agent orchestration. Rule LC-PROM-001 requires prompt sanitization before tool calls. The Agent Card documents which sanitization mechanisms are applied, which PII patterns are detected, and which tools receive sanitized vs. unsanitized inputs. Rule addressing tool authorization documents in the Agent Card which tools the agent can access, under what conditions, and with what approval workflows.

Agent Card Sections and Corresponding Rules:

Agent Card Section	Rules Addressed	Evidence Provided
Agent Architecture	LC-TRANS-001, System design transparency	Orchestration framework, reasoning loop, memory structure
Tool Inventory	LC-IP-001, Authorization controls	Available tools, permissions, rate limits, approval workflows
Input Processing	LC-PROM-001, LC-PROM-003, KSERVE-2.5.1	Prompt sanitization, PII redaction, injection detection
Safety Guardrails	KSERVE-2.5.2, MLF-ROBUST-001	Behavioral constraints, prohibited operations, escalation rules
Action Validation	LC-ROB-001, Tool authorization	Pre-execution checks, confirmation requirements, rollback procedures
Monitoring & Logging	LC-TRANS-002, Audit requirements	Telemetry data, decision logs, action traces, security events

Agent Cards supplement Model Cards by documenting the control layer around agentic systems. Both are essential for compliance: the Model Card details the underlying

model, and the Agent Card documents its constraints and monitoring within the agentic framework.

9.4.3 Integrating Cards into Compliance Workflows

Model and Agent Cards must be integrated into automated compliance workflows, not treated as static artifacts. Policy engine rule enforcement should automatically generate or update card content. This tight integration ensures cards are accurate, living documents reflecting current rules.

Workflow Integration Pattern:

1. **Development Phase:**
    - Developer trains model, MLflow logs metrics automatically
    - Policy engine validates metrics against thresholds (rules enforced)
    - MLflow generates Model Card from logged metrics
    - Model Card includes pass/fail status for each rule
2. **Deployment Phase:**
    - Model deployment request includes Model Card reference
    - Kubernetes admission controller validates card completeness (rule enforcement)
    - Inference Service deploys with card metadata as annotations
    - Agent orchestration wraps model with guardrails documented in Agent Card
3. **Runtime Phase:**
    - Guardrails enforce policies documented in Agent Card
    - Monitoring systems track metrics specified in cards
    - Alerting triggers when metrics violate thresholds from rules
    - Cards update automatically with production metrics
4. **Audit Phase:**
    - Auditor requests compliance evidence for specific rules
    - System retrieves Model Cards and Agent Cards for relevant systems
    - Cards provide evidence trail: rule → enforcement → metrics → pass/fail
    - Automated report generation maps rules to card evidence

This integration pattern ensures cards are not just documentation but active participants in the compliance workflow. Cards drive policy enforcement decisions, provide runtime monitoring targets, and supply audit evidence—all while remaining synchronized with the systems they describe.

10 Future Directions and Emerging Challenges

Chapter 9 delves into the dynamic landscape of AI safety by exploring future directions and emerging challenges that demand proactive attention from organizations and regulators. It highlights the complex risks introduced by agentic AI systems, which can autonomously plan and execute actions, necessitating new security paradigms to prevent unintended tool use and goal misalignment. Furthermore, the chapter examines the rapid evolution of the global regulatory environment, particularly focusing on the implications of frameworks like the European Union AI Act and the challenge of establishing globally consistent safety programs. Finally, it addresses the increasingly sophisticated threat vectors targeting AI, including advanced prompt injection techniques and supply chain attacks, underscoring the continuous need for robust defenses to maintain the integrity and security of deployed AI systems.

10.1 Agentic AI Systems

The emergence of agentic AI systems, which can autonomously plan, execute actions, and use external tools, represent a major shift and introduce unique risks beyond generative AI. Challenges include unintended tool use, privilege escalation through chained actions, and goal misalignment. Traditional security methods are less effective as agents explore vast action spaces. Organizations must implement comprehensive monitoring, define clear boundaries, and design human-review fallback mechanisms.

Multi-agent systems add complexity, risking probabilistic behaviors, instability from adversarial dynamics, and resource exhaustion. Deploying organizations need careful agent isolation, inter-agent communication monitoring, anomaly-detecting circuit breakers, and extensive simulation testing before production.

10.2 Regulatory Evolution

The regulatory landscape for AI continues to evolve rapidly as governments worldwide grapple with balancing innovation and risk management. The European Union AI Act establishes risk-based requirements with strict obligations for high-risk systems including transparency, human oversight, robustness, and accuracy. Organizations deploying AI in European markets must classify their systems, implement appropriate technical and organizational measures, maintain comprehensive documentation, and undergo conformity assessment for high-risk applications. The extraterritorial reach of the AI Act means even non-European organizations may face compliance obligations when their AI systems are used in the EU.

While international coordination exists, regional differences persist. A key challenge is developing global AI safety programs that satisfy varied regulations. Machine-readable

frameworks like OSCAL can help by documenting how core safety controls meet diverse requirements, adjusted by deployment location.

10.3 Emerging Threat Vectors

As AI systems advance and training and inference infrastructure gets cheaper/easier to use, threat actors use increasingly sophisticated attacks like multi-turn and indirect prompt injection to subvert system behavior. Organizations must use robust input sanitization, maintain clear content boundaries, and monitor for injection attempts.

AI supply chain attacks compromise training data, model weights, or dependencies via poisoned data, backdoored models, or compromised libraries, exploiting the community's trust. Organizations should cryptographically verify external resources, scan dependencies, and maintain internal mirrors to prevent dependency confusion.

Model extraction and stealing aim to recreate proprietary models through carefully crafted queries. Defenses include rate limiting, detecting structured querying, watermarking outputs, and terms of service to balance model accessibility with preventing unauthorized extraction.

11 Conclusion

Chapter 10 summarizes the whitepaper's systematic approach to AI safety, detailing key practices and frameworks for risk management and trustworthy AI. It emphasizes robust organizational capabilities, cross-functional collaboration, and concludes by urging the adoption of continuous, automated compliance practices.

11.1 Summary of Key Practices

Comprehensive AI safety needs systematic attention across the entire AI lifecycle and technical stack. This paper offers frameworks—lifecycle phases and stack layers—to understand risks and apply targeted controls, enabling organizations to build trustworthy, compliant, and resilient AI systems. Transitioning to machine-readable compliance via OSCAL Component Definitions automates compliance, shifting it from periodic audits to continuous verification integrated with DevSecOps workflows. Investing in OSCAL-based automation provides efficiency, reduces audit costs, and ensures more consistent compliance than manual processes.

11.2 Organizational Capabilities

Developing trustworthy AI demands more than technical controls; it requires organizational capabilities across people, processes, and technology. Cross-functional collaboration is crucial, involving AI engineers, security teams, compliance staff, and business stakeholders. Engineers grasp technical limits, security teams counter

vulnerabilities, compliance staff ensure regulatory alignment, and business stakeholders set risk tolerance. Effective AI safety necessitates breaking down silos and establishing shared understanding among these groups.

11.3 Call to Action

Organizations must adapt these AI safety principles to their context, risk tolerance, and regulations. Practitioners should assess their current AI safety posture using the provided frameworks to identify gaps. Prioritize efforts based on risk, focusing on high-stakes applications and weak controls. Implement incrementally; comprehensive AI safety requires sustained effort.

Finally, we will engage with the AI safety community (e.g., LF AI & Data Foundation, CNCF, NIST) to share experiences and contribute to evolving best practices, recognizing that collective intelligence accelerates ecosystem maturity.

References and Resources

- IBM AI Risk Atlas. https://www.ibm.com/ai-risk-atlas
- IBM Risk Atlas Nexus. https://github.com/IBM/risk-atlas-nexus
- NIST AI Risk Management Framework. https://www.nist.gov/itl/ai-risk-management-framework
- NIST SP 800-53 Rev 5. https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final
- OSCAL v1.2.2. https://pages.nist.gov/OSCAL-Reference/models/v1.2.2/
- EU Artificial Intelligence Act. https://artificialintelligenceact.eu/
- ISO/IEC 42001:2023 - AI Management Systems. https://www.iso.org/standard/81230.html
- CNCF Technology Radar: AI Inferencing. October 2025. https://www.cncf.io/wp-content/uploads/2025/11/cncf_report_techradar_111025a.pdf
- LF AI & Data Security and Compliance. https://github.com/lfai/security-and-compliance
- TrustyAI Documentation. https://trustyai.org/
- Model Context Protocol Specification. Anthropic. https://modelcontextprotocol.io
- Australia Voluntary AI Safety Standard. Department of Industry, 2024 https://www.industry.gov.au/publications/voluntary-ai-safety-standard
- South Korean AI Basic Law https://aibasicact.kr/
- Canada AIDA (proposed) http://ised-isde.canada.ca/site/innovation-better-canada/en/artificial-intelligence-and-data-act

Brazil AI Bill No. 2338/2023 (proposed) https://www.dataprivacybr.org/en/the-artificial-intelligence-legislation-in-brazil-technical-analysis-of-the-text-to-be-voted-on-in-the-federal-senate-plenary/

Policy Engine Documentation

- Open Policy Agent: https://www.openpolicyagent.org/docs/
- Kyverno: https://kyverno.io/docs/
- NeMo Guardrails: https://github.com/NVIDIA/NeMo-Guardrails
- Guardrails AI: https://docs.guardrailsai.com/
- Terraform Sentinel: https://docs.hashicorp.com/sentinel/

Model Card and Agent Card Resources

- Model Cards Paper: Mitchell et al., 2019 (https://arxiv.org/abs/1810.03993)
- Hugging Face Model Cards: https://huggingface.co/docs/hub/model-cards
- Model Card Toolkit: https://github.com/tensorflow/model-card-toolkit
- Agent Cards (emerging standard): https://github.com/a2aproject/A2A

Appendix A: Quick Start Guide

This guide provides a systematic approach for organizations to implement AI safety and compliance practices using the frameworks, rules, and tools described in this white paper. The guide assumes you have completed an initial review of this document and are ready to begin practical implementation.

Prerequisites

Before beginning implementation, ensure your organization has the following foundational elements in place:

Technical Prerequisites

- AI workloads deployed on Kubernetes or similar container orchestration platform
- Access to AI development tools (MLflow, KServe, LangChain, or similar frameworks)
- Existing observability infrastructure (metrics, logging, tracing)
- Source control for infrastructure-as-code (Terraform, CloudFormation, or similar)
- CI/CD pipelines for model deployment

Knowledge Prerequisites

- Familiarity with NIST AI Risk Management Framework (read NIST AI RMF 1.0)
- Understanding of your organization's AI risk profile and regulatory requirements
- Basic knowledge of policy-as-code concepts (OPA, Kyverno, or similar)
- Awareness of AI safety risks relevant to your use cases (review Chapter 3: Risk Atlas Nexus)

Organizational Prerequisites

- Executive sponsorship for AI governance initiative
- Cross-functional team including AI/ML engineers, security, compliance, and legal
- Defined roles and responsibilities for AI safety (see Chapter 10.2)
- Budget allocated for tooling and training

This 16-week quick start program is organized in four phases and the detailed action plan is presented next.

Phase	Duration
Phase 1: Assessment and Planning	Weeks 1-2
Phase 2: Pilot Implementation	Weeks 3-6
Phase 3: Expand Coverage	Weeks 7-12

Phase	Duration
Phase 4: Operationalize and Scale	Weeks 13-16

Phase 1: Assessment and Planning (Weeks 1-2)

Begin by understanding your current AI landscape and identifying the highest-priority risks to address.

Step 1: Map Your AI Stack (Week 1)

Inventory which layers of the AI stack your organization uses. Reference Chapter 4 for layer definitions.

Action Items:

- Document which AI frameworks you use (PyTorch, TensorFlow, LangChain, etc.)
- Identify model serving infrastructure (KServe, vLLM, BentoML, TorchServe, etc.)
- List data pipeline tools (Apache Airflow, Kafka, Spark, Great Expectations, etc.)
- Identify ML orchestration platforms (MLflow, Tekton, DVC, etc.)
- Document agent frameworks if applicable (LangChain, AutoGen, MCP Gateway)
- Map tools to stack layers using the taxonomy in Chapter 4

Deliverable: AI Stack Inventory document mapping your tools to Layers 3-7

Step 2: Identify Priority Risks (Week 1)

Use the IBM AI Risk Atlas Nexus taxonomy (Chapter 3) to identify which risk categories apply to your AI systems.

Action Items:

- Review Input Risks: Do you handle sensitive data? Are you vulnerable to data poisoning?
- Review Inference Risks: Are you exposed to prompt injection, jailbreaking, or evasion attacks?
- Review Output Risks: Could your models generate harmful content, hallucinations, or biased outputs?
- Review Non-Technical Risks: Are governance, transparency, and accountability processes defined?
- Review Use Context Risks: What are deployment-specific safety concerns for your use cases?
- Prioritize risks based on likelihood and impact using your risk assessment framework

Deliverable: Risk Assessment Matrix ranking top 10 AI risks for your organization

Step 3: Download Compliance Rule Spreadsheets (Week 2)

Access the pre-defined compliance rules from the LF AI & Data Security and Compliance Working Group repository.

Repository: https://github.com/lfai/security-and-compliance/tree/main/subgroups/risk-and-compliance/content

Action Items:

- Download KServe rules if you use KServe for model serving
- Download LangChain rules if you use LangChain or similar agent frameworks
- Download MLflow rules if you use MLflow for experiment tracking
- Download MCP Gateway rules if you use MCP for agent tool access
- Download AI Lifecycle rules (applies to all organizations)
- Review rules and identify which apply to your stack and risk profile
- Map rules to NIST AI RMF controls relevant to your compliance requirements

Deliverable: Selected rules spreadsheet (filtered to your applicable rules only)

Step 4: Select Policy Enforcement Tools (Week 2)

Choose policy engines appropriate for the layers where you need enforcement. Reference Chapter 8 for policy engine selection guidance.

Stack Layer	Policy Engine Type	Recommended Tools
Layer 7: Applications	AI Guardrails	NeMo Guardrails, Guardrails AI, or Lakera Guard
Layer 6: ML Orchestration	Workflow validation	MLflow policy hooks, Tekton admission
Layer 5: Model Serving	Inference validation	KServe transformers, model validators
Layer 4: Data Pipeline	Data quality	Great Expectations, dbt tests
Layer 3: Platform (Infra)	Infrastructure policy	OPA, Kyverno, or CloudCustodian

Decision Criteria:

- Start with Layer 7 (AI Guardrails) - highest immediate impact on safety
- Add Layer 3 (Infrastructure Policy) if deploying to Kubernetes
- Add Layer 4 (Data Quality) if training models in-house
- Defer Layer 5-6 unless specific requirements identified

Deliverable: Policy engine selection document with justification

Phase 2: Pilot Implementation (Weeks 3-6)

Implement a focused pilot covering your highest-priority risks with a subset of rules. This validates the approach before full-scale rollout.

Step 5: Deploy AI Guardrails (Weeks 3-4)

Begin with Layer 7 AI guardrails to prevent immediate safety risks at the inference path.

If using NeMo Guardrails:

- Install NeMo Guardrails: pip install nemoguardrails
- Define rails configuration file (config.yml) with input/output rails
- Implement prompt injection detection rail (reference MCP-INJ-001)
- Implement harmful output filtering rail (reference MLF-HARM-001)
- Configure topic boundaries and behavioral constraints
- Integrate with your LLM inference endpoint
- Test with known attack prompts from jailbreak databases

Success Metrics:

Prompt injection detection rate >95%, Harmful content detection rate >99%, False positive rate <2%

Deliverable: AI guardrails deployed on pilot LLM application with metrics dashboard

Step 6: Implement Infrastructure Policy (Weeks 4-5)

Deploy OPA or Kyverno policies to enforce governance controls on model deployments.

If using Kyverno (Kubernetes):

- Install Kyverno in your Kubernetes cluster: kubectl apply -f kyverno.yaml
- Create policy requiring model cards for InferenceService deployments (reference KSERVE-1.3.1)
- Create policy enforcing resource limits on GPU workloads
- Create policy requiring cost allocation labels on all AI workloads
- Test policies with sample InferenceService manifests
- Deploy policies in audit mode initially, then shift to enforce mode
- Monitor policy violations and adjust thresholds

Success Metrics:

Policy enforcement rate 100%, Documentation completeness >95%, Zero unauthorized deployments

Deliverable: Kubernetes admission policies enforcing model documentation and resource governance

Step 7: Enable Monitoring and Observability (Week 6)

Instrument your AI systems to track compliance metrics defined in the rule spreadsheets.

Action Items:

- Set up metrics collection for guardrail enforcement (prometheus/grafana)
- Configure MLflow to log fairness metrics during training (reference MLF-FAIR-001)
- Enable audit logging for all model inferences (reference OPS-004)
- Create dashboards showing policy enforcement metrics by rule ID
- Set up alerts for threshold violations (e.g., fairness degradation >5%)
- Establish weekly review process for compliance metrics

Deliverable: Compliance dashboard tracking metrics from implemented rules

Phase 3: Expand Coverage (Weeks 7-12)

Scale successful pilot patterns to cover additional rules, systems, and lifecycle phases.

Step 8: Implement Data Pipeline Controls (Weeks 7-9)

Add data quality and fairness validation to training data pipelines.

If using Great Expectations:

- Install Great Expectations: pip install great_expectations
- Initialize GE project in your data pipeline repository
- Define expectations for PII detection (reference DATA-COL-002)
- Define expectations for demographic representation (reference DATA-COL-001)
- Define expectations for data integrity (checksums) (reference DATA-COL-005)
- Integrate expectations into Airflow/pipeline DAGs
- Configure checkpoint validation before training jobs
- Generate data quality reports for compliance audit

Success Metrics:

Data validation pass rate >98%, PII detection rate >95%, Demographic representation >5% per group

Step 9: Implement Lifecycle Controls (Weeks 9-11)

Apply controls across training, evaluation, deployment, and monitoring phases using AI Lifecycle rules.

Training Phase:

- Log fairness metrics during training (TRAIN-003)

- Validate train/test separation (TRAIN-002)
- Test for memorization (TRAIN-001)
- Document all hyperparameters (TRAIN-005)

Evaluation Phase:

- Test across demographic subgroups (EVAL-001)
- Conduct adversarial testing (EVAL-002)
- Measure hallucination rates (EVAL-003)
- Validate explainability (EVAL-005)

Deployment Phase:

- Require model cards (DEPLOY-001)
- Implement input validation (DEPLOY-002)
- Deploy safety guardrails (DEPLOY-003)
- Enable rollback capability (DEPLOY-004)
- Require human approval (DEPLOY-005)

Operations Phase:

- Monitor for drift (OPS-001)
- Monitor fairness degradation (OPS-002)
- Detect feedback loops (OPS-003)
- Maintain audit logs (OPS-004)
- Test incident response (OPS-005)

Step 10: Generate Model Cards and Agent Cards (Week 12)

Automate generation of compliance documentation from enforced rules and logged metrics.

Action Items:

- Use Model Card Toolkit to generate cards from MLflow metrics
- Populate Model Card sections from rule evidence (reference Chapter 9.4.1)
- Create Agent Cards for agentic systems documenting tool access controls
- Link Model Cards to InferenceService deployments (Kubernetes annotations)
- Publish cards to model registry for stakeholder access
- Establish process for card updates when models are retrained

Deliverable: Automated Model Card generation integrated into deployment pipeline

Phase 4: Operationalize and Scale (Weeks 13-16)

Transition from pilot to production-ready compliance program with continuous improvement.

Step 11: Implement OSCAL Automation (Weeks 13-14)

Represent compliance posture as machine-readable OSCAL artifacts for automated reporting. Reference Chapter 7 for OSCAL guidance.

Action Items:

- Create OSCAL Component Definition for each AI system
- Map implemented rules to NIST 800-53 or other framework controls
- Document control implementations with evidence from metrics
- Use compliance-trestle or similar tool for OSCAL authoring
- Generate System Security Plans (SSPs) from Component Definitions
- Automate OSCAL artifact updates when rules are modified
- Integrate OSCAL validation into CI/CD pipelines

Deliverable: OSCAL Component Definitions for all production AI systems

Step 12: Establish Continuous Compliance (Weeks 15-16)

Create processes for ongoing compliance monitoring, rule updates, and audit readiness.

Action Items:

- Schedule weekly compliance metrics review meetings
- Establish quarterly rule coverage assessment (are new rules needed?)
- Create runbooks for responding to compliance metric violations
- Define escalation paths for safety incidents
- Schedule annual audit simulation exercises
- Contribute rule improvements back to LFAI community repository
- Monitor regulatory changes and update rules accordingly

Deliverable: Compliance operations playbook with defined roles, responsibilities, and procedures

Success Criteria and Metrics

Evaluate the success of your AI safety and compliance implementation using these criteria:

Metric Category	Success Threshold	Measurement Method
Rule Coverage	>80% of applicable rules enforced	Count enforced rules / Total applicable rules
Policy Enforcement	100% enforcement rate	Zero bypassed policies, all violations logged
Safety Metrics	All thresholds met	Prompt injection >95%, Harm detection >99%, Drift alerts <1%

Metric Category	Success Threshold	Measurement Method
Documentation	100% models have cards	All deployed models have Model Cards or Agent Cards
Audit Readiness	<4 hours evidence retrieval	Time to produce compliance evidence for random audit
Incident Response	<4 hours mean resolution	Average time from safety incident to mitigation

Common Pitfalls to Avoid

Pitfall 1: Boiling the Ocean

Attempting to implement all 127 rules across 5 spreadsheets simultaneously. Start with 10-15 highest-priority rules covering your top risks. Expand coverage incrementally every quarter.

Pitfall 2: Policy Without Enforcement

Creating policy documents and rules spreadsheets without implementing actual policy engines. Every rule must have corresponding automated checks via OPA, Kyverno, NeMo, or other enforcement mechanisms.

Pitfall 3: Infrastructure Before AI Controls

Implementing Kubernetes policies and IaC scanning while ignoring AI-specific risks like prompt injection, hallucinations, and fairness. Start with Layer 7 AI guardrails for immediate safety impact.

Pitfall 4: Ignoring Model Cards

Treating Model Cards as optional documentation rather than compliance evidence. Model Cards should be auto-generated from rule enforcement metrics and required before deployment (DEPLOY-001).

Pitfall 5: Set and Forget

Deploying guardrails and policies without ongoing monitoring and tuning. Establish weekly metric reviews and quarterly rule coverage assessments to maintain effectiveness.

Next Steps After Quick Start

After completing this 16-week quick start program, consider these advanced topics:

- Contribute rule improvements to LFAI Security & Compliance Working Group

- Integrate compliance metrics into executive dashboards and OKRs
- Expand to additional AI frameworks not covered in pilot (e.g., new model serving platforms)
- Implement continuous compliance testing in CI/CD pipelines
- Conduct red team exercises against deployed guardrails
- Publish case studies and lessons learned to LFAI community
- Pursue external audit or certification (ISO 42001, SOC 2 AI controls)
- Scale compliance program to cover emerging AI capabilities (multimodal, agentic systems)

NOTE: The final text of this white paper has the AI generated text edited and reviewed by the workgroup.

Appendix B: Tool Ecosystem and Resources

This appendix provides comprehensive references to tools, repositories, standards, and community resources for implementing AI safety and compliance practices. Some tools mentioned below may be commercial.

Compliance Rule Repositories

LF AI & Data Security and Compliance Working Group

The primary repository for AI compliance rules derived from NIST AI RMF and IBM AI Risk Atlas Nexus taxonomy.

Repository: https://github.com/lfai/security-and-compliance/tree/main/subgroups/risk-and-compliance/content

Available rule sets (pending update)

- KServe Rules (18 rules): Platform-level controls for model serving infrastructure covering data validation, privacy, robustness, and transparency
- LangChain Rules (20 rules): Framework controls for agent orchestration, document loading, prompt handling, and RAG pipelines
- MLflow Rules (30 rules): Lifecycle management controls for experiment tracking, model registry, training data governance, and deployment
- MCP Gateway Rules (30 rules): Agent-specific controls for tool access authorization, prompt injection detection, action validation, and observability
- AI Lifecycle Rules (29 rules): Cross-cutting controls spanning data collection, training, evaluation, deployment, operations, and retraining phases

Each rule includes: Rule ID, description, NIST AI RMF mapping, associated metrics, calculation method, threshold values, and HuggingFace metric alignment.

Policy Engines

Infrastructure Policy Engines

Open Policy Agent (OPA)

- Purpose: General-purpose policy engine using Rego policy language
- Use Cases: Kubernetes admission control, API authorization, infrastructure validation
- Website: https://www.openpolicyagent.org
- Documentation: https://www.openpolicyagent.org/docs/latest/
- Getting Started: https://www.openpolicyagent.org/docs/latest/#running-opa
- License: Apache 2.0

Kyverno

- Purpose: Kubernetes-native policy engine using YAML policies
- Use Cases: Pod security, resource validation, configuration management, policy reporting
- Website: https://kyverno.io
- Documentation: https://kyverno.io/docs/
- Getting Started: https://kyverno.io/docs/installation/
- License: Apache 2.0

CloudCustodian

- Purpose: Cloud resource governance and compliance automation
- Use Cases: Multi-cloud policy enforcement, cost optimization, security compliance
- Website: https://cloudcustodian.io
- Documentation: https://cloudcustodian.io/docs/
- Repository: https://github.com/cloud-custodian/cloud-custodian
- License: Apache 2.0

Infrastructure-as-Code Policy Engines

Terraform Sentinel

- Purpose: Policy-as-code for Terraform plans (HashiCorp commercial product)
- Use Cases: Infrastructure compliance, cost controls, security policies before deployment
- Website: https://www.terraform.io/cloud-docs/policy-enforcement
- Documentation: https://docs.hashicorp.com/sentinel
- License: Commercial (part of Terraform Cloud/Enterprise)

Checkov

- Purpose: Static analysis for infrastructure-as-code security and compliance
- Use Cases: Terraform, CloudFormation, Kubernetes YAML scanning
- Website: https://www.checkov.io
- Documentation: https://www.checkov.io/documentation.html
- Repository: https://github.com/bridgecrewio/checkov
- License: Apache 2.0

Terrascan

- Purpose: Static code analyzer for infrastructure-as-code
- Use Cases: Terraform, Kubernetes, Helm, Kustomize policy validation
- Repository: https://github.com/tenable/terrascan
- Documentation: https://runterrascan.io/docs/
- License: Apache 2.0

AI Inference Guardrails

NVIDIA NeMo Guardrails

- Purpose: Programmable guardrails for conversational AI systems
- Use Cases: Input validation, output filtering, dialog flow control, fact-checking
- Repository: https://github.com/NVIDIA/NeMo-Guardrails
- Documentation: https://github.com/NVIDIA/NeMo-Guardrails/tree/main/docs
- Getting Started: https://github.com/NVIDIA/NeMo-Guardrails#getting-started
- License: Apache 2.0

Guardrails AI

- Purpose: Python framework for building reliable AI applications with validation
- Use Cases: Output validation, structured data extraction, LLM response validation
- Website: https://www.guardrailsai.com
- Documentation: https://docs.guardrailsai.com
- Repository: https://github.com/guardrails-ai/guardrails
- License: Apache 2.0

Lakera Guard

- Purpose: Enterprise AI security platform for prompt injection and jailbreak detection (commercial product)
- Use Cases: LLM API protection, prompt firewall, content moderation
- Website: https://www.lakera.ai/guard
- Documentation: https://platform.lakera.ai/docs
- License: Commercial (free tier available)

Llama Guard

- Purpose: Content moderation classifier for LLM inputs and outputs
- Use Cases: Safety classification, policy violation detection, content filtering
- Repository: https://github.com/meta-llama/PurpleLlama/tree/main/Llama-Guard
- Model: https://huggingface.co/meta-llama/LlamaGuard-7b
- License: Meta Llama 3 Community License

AI Stack Tools by Layer

Layer 7: Applications and APIs

LangChain

- Purpose: Framework for building LLM applications with chains, agents, and RAG
- Repository: https://github.com/langchain-ai/langchain

- Documentation: https://python.langchain.com/docs/
- License: MIT

AutoGen

- Purpose: Multi-agent conversation framework for building autonomous agents
- Repository: https://github.com/microsoft/autogen
- Documentation: https://microsoft.github.io/autogen/
- License: MIT

MCP Gateway (ContextForge)

- Purpose: Gateway for agent tool access control using Model Context Protocol
- Use Cases: Tool authorization, rate limiting, audit logging for agent actions
- Note: Reference implementation for MCP security controls
- Protocol: https://modelcontextprotocol.io

Layer 6: ML Orchestration

MLflow

- Purpose: ML lifecycle platform for experiment tracking, model registry, deployment
- Website: https://mlflow.org
- Documentation: https://www.mlflow.org/docs/latest/index.html
- Repository: https://github.com/mlflow/mlflow
- License: Apache 2.0

Tekton

- Purpose: Kubernetes-native CI/CD pipeline framework
- Website: https://tekton.dev
- Documentation: https://tekton.dev/docs/
- Repository: https://github.com/tektoncd/pipeline
- License: Apache 2.0

DVC (Data Version Control)

- Purpose: Version control for ML models and datasets
- Website: https://dvc.org
- Documentation: https://dvc.org/doc
- Repository: https://github.com/iterative/dvc
- License: Apache 2.0

TensorBoard

- Purpose: Visualization toolkit for ML experiments
- Website: https://www.tensorflow.org/tensorboard

- Documentation: https://www.tensorflow.org/tensorboard/get_started
- Repository: https://github.com/tensorflow/tensorboard
- License: Apache 2.0

Layer 5: Model Frameworks and Inference Serving

KServe

- Purpose: Serverless inference platform on Kubernetes
- Website: https://kserve.github.io/website/
- Documentation: https://kserve.github.io/website/latest/
- Repository: https://github.com/kserve/kserve
- License: Apache 2.0

vLLM

- Purpose: High-throughput LLM inference engine with PagedAttention
- Repository: https://github.com/vllm-project/vllm
- Documentation: https://docs.vllm.ai/
- License: Apache 2.0

BentoML

- Purpose: Framework for building and deploying ML services
- Website: https://www.bentoml.com
- Documentation: https://docs.bentoml.com
- Repository: https://github.com/bentoml/BentoML
- License: Apache 2.0

PyTorch

- Purpose: Deep learning framework for model development
- Website: https://pytorch.org
- Documentation: https://pytorch.org/docs/
- Repository: https://github.com/pytorch/pytorch
- License: BSD

Milvus

- Purpose: Vector database for similarity search and embeddings
- Website: https://milvus.io
- Documentation: https://milvus.io/docs
- Repository: https://github.com/milvus-io/milvus
- License: Apache 2.0

Layer 4: Data Pipeline and Orchestration

Apache Airflow

- Purpose: Workflow orchestration platform for data pipelines
- Website: https://airflow.apache.org
- Documentation: https://airflow.apache.org/docs/
- Repository: https://github.com/apache/airflow
- License: Apache 2.0

Great Expectations

- Purpose: Data quality validation framework
- Website: https://greatexpectations.io
- Documentation: https://docs.greatexpectations.io/
- Repository: https://github.com/great-expectations/great_expectations
- License: Apache 2.0

Delta Lake

- Purpose: ACID storage layer for data lakes with versioning
- Website: https://delta.io
- Documentation: https://docs.delta.io/
- Repository: https://github.com/delta-io/delta
- License: Apache 2.0

Apache Kafka

- Purpose: Distributed event streaming platform
- Website: https://kafka.apache.org
- Documentation: https://kafka.apache.org/documentation/
- License: Apache 2.0

Standards and Compliance Frameworks

NIST AI Risk Management Framework (AI RMF)

- Version: AI RMF 1.0 (January 2023)
- Website: https://www.nist.gov/itl/ai-risk-management-framework
- Download: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
- Playbook: https://airc.nist.gov/AI_RMF_Knowledge_Base/Playbook
- Purpose: Voluntary framework for managing AI risks across lifecycle

NIST OSCAL (Open Security Controls Assessment Language)

- Version: 1.2.2 (current with mapping model)
- Website: https://pages.nist.gov/OSCAL/
- Documentation: https://pages.nist.gov/OSCAL/documentation/
- Repository: https://github.com/usnistgov/OSCAL
- Purpose: Machine-readable representation of security controls and compliance

ISO/IEC 42001:2023

- Title: Artificial Intelligence Management System
- Published: December 2023
- Website: https://www.iso.org/standard/81230.html
- Purpose: International standard for AI management systems and governance
- Key Controls: Data governance (6.2.6), Risk management (6.1), Lifecycle management (7.2)

EU AI Act

Following amendments to the EU AI Act, the application dates for certain obligations have been postponed, allowing sufficient time for implementation. The most important dates relating to high-risk AI systems are:

- 2 December 2027 for AI systems listed in Annex III of the AI Act
- 2 August 2028 AI systems embedded in regulated products covered by the EU sectoral legislation listed in Annex I

- Status: Adopted by EU Parliament (March 2024), enforcement began to apply in stages from February 2025
- Timeline: https://artificialintelligenceact.eu/implementation-timeline/
- Website: https://artificialintelligenceact.eu
- Official Text: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689
- Purpose: Risk-based regulation of AI systems in EU market
- Key Requirements: High-risk AI documentation, transparency, human oversight, accuracy

Model Documentation Standards

Model Cards

- Original Paper: Mitchell et al., 2019 - https://arxiv.org/abs/1810.03993
- Hugging Face Implementation: https://huggingface.co/docs/hub/model-cards
- TensorFlow Toolkit: https://github.com/tensorflow/model-card-toolkit
- Purpose: Structured documentation of ML model characteristics, performance, and limitations

Agent Cards

- Purpose: Extension of Model Cards for agentic AI systems
- Documents: Tool access, behavioral constraints, safety guardrails, action validation
- Status: Emerging standard under community development
- Reference: Chapter 8.4.2 of this white paper

Community Resources and Working Groups

LF AI & Data Foundation

- Website: https://lfaidata.foundation
- Security & Compliance WG: Bi-weekly meetings, mailing list
- Meeting Schedule: https://wiki.lfaidata.foundation/display/DL/Security+and+Compliance
- How to Join: https://lfaidata.foundation/join/
- Slack: https://lfaifoundation.slack.com (channel: #security-compliance)

OSCAL Foundation

- Website: https://oscal-foundation.org
- Purpose: Community development of OSCAL standards and tooling
- Meetings: Monthly community calls
- Mailing List: oscal-foundation@lists.lfai.foundation
- Slack: Join via LFAI Slack workspace

NIST OSCAL Community

- Mailing List: oscal@nist.gov
- GitHub Discussions: https://github.com/usnistgov/OSCAL/discussions
- Community Calls: Monthly (check NIST OSCAL website)
- OSCAL Content Repository: https://github.com/usnistgov/oscal-content

AI Alliance

- Website: https://thealliance.ai
- Risk Atlas Nexus: Community taxonomy for AI risks
- Working Groups: Safety, trust, policy
- Members: IBM, Meta, NASA, NSF, and 50+ organizations

Training and Educational Resources

NIST AI RMF Training

- AI RMF Playbook: https://airc.nist.gov/AI_RMF_Knowledge_Base/Playbook
- NIST Trustworthy AI Resource Center: https://airc.nist.gov
- AI RMF Crosswalks: Maps to ISO 42001, NIST CSF, CIS Controls
- Free online training: Available through NIST website

OSCAL Tutorials

- Official Tutorials: https://pages.nist.gov/OSCAL/learn/tutorials/
- Compliance-Trestle Docs: https://oscal-compass.github.io/compliance-trestle/
- Video Tutorials: NIST YouTube channel - search "OSCAL"
- Workshops: OSCAL Foundation hosts quarterly workshops

Policy-as-Code Training

- OPA Academy: https://academy.styra.com
- Kyverno Learning: https://kyverno.io/docs/writing-policies/
- Policy as Code Book: https://www.policyascode.io
- CNCF Security TAG: https://tag-security.cncf.io

Research Papers and Publications

Foundational AI Safety Research

- Model Cards: Mitchell et al. (2019) - https://arxiv.org/abs/1810.03993
- Datasheets for Datasets: Gebru et al. (2021) - https://arxiv.org/abs/1803.09010
- AI Safety Benchmarks: HELM (Liang et al., 2023) - https://crfm.stanford.edu/helm/
- Adversarial Robustness: Carlini & Wagner (2017) - https://arxiv.org/abs/1608.04644
- Red Teaming LLMs: Perez et al. (2022) - https://arxiv.org/abs/2202.03286

AI Governance and Policy

- NIST AI RMF 1.0: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
- EU AI Act Analysis: https://artificialintelligenceact.eu/assessment/
- ISO 42001 Guidance: https://www.iso.org/standard/81230.html
- Responsible AI Practices: Google - https://ai.google/responsibility/responsible-ai-practices/

Additional Specialized Tools

Fairness and Bias Testing

- Fairlearn: https://fairlearn.org - Fairness assessment and mitigation
- AI Fairness 360: https://aif360.mybluemix.net - IBM fairness toolkit
- What-If Tool: https://pair-code.github.io/what-if-tool/ - Model understanding

Explainability and Interpretability

- SHAP: https://github.com/slundberg/shap - Shapley value explanations
- LIME: https://github.com/marcotcr/lime - Local interpretable explanations
- InterpretML: https://interpret.ml - Microsoft interpretability toolkit
- Captum: https://captum.ai - PyTorch model interpretation

Red Teaming and Security Testing

- PyRIT: https://github.com/Azure/PyRIT - Microsoft red team framework
- Garak: https://github.com/leondz/garak - LLM vulnerability scanner
- Purple Llama: https://github.com/meta-llama/PurpleLlama - Meta safety tools
- PromptInject: https://github.com/agencyenterprise/PromptInject - Injection testing

Getting Help and Support

For questions, contributions, or support implementing AI safety and compliance practices:

LFAI Risk and Compliance Workgroup
This group focuses on security and policy frameworks for AI-enabled applications.

- **Mailing List:** Join the security-and-compliance-work-group@lists.lfaidata.foundationlist for announcements and technical discussions.
- **Slack Channel:** Engage in real-time via the `#security-and-compliance-work-group` channel on the LF AI & Data Slack.
- **Meetings:** The group meets **biweekly on Tuesdays at 10:00 AM ET**. Registration and details are maintained on their LF AI & Data Wiki page.
- **Official Page:** LFAI Work Groups Overview.

OSCAL-Compass CNCF community
OSCAL-Compass is a CNCF Sandbox project (includes Trestle) providing tools for "Compliance-as-Code."

- **GitHub Community Hub:** The oscal-compass/community repository serves as the central coordination point for governance and community-wide collaboration.
- **Project Documentation:** Technical details on Agile Authoring and Trestle are available at oscal.io.
- **CNCF Slack:** Look for the `#oscal-compass-trestle-agileauthoring-c2p` channel on the CNCF Slack for developer and user support.
- **Official CNCF Project Page:** OSCAL-Compass on CNCF.

OSCAL Foundation TAG community

The OSCAL Foundation supports the broader ecosystem and standardization of NIST OSCAL.

- **Technical Working Group (TWG):** Join the technical development efforts by subscribing to technical-working-group+subscribe@oscalfoundationlists.org.
- **General Membership:** Subscribe to news and announcements via news+subscribe@oscalfoundationlists.org or the OSCAL Foundation Get Involved page.
- **Engagement Working Group:** Centralizes efforts to broaden OSCAL adoption; access is available through the OSCAL Foundation home

Contributing to Rule Repositories

- Fork repository: https://github.com/lfai/security-and-compliance
- Add or improve rules in CSV format following existing structure
- Submit pull request with description of changes

- Attend working group meeting to discuss contributions
- All contributions welcome: new rules, metrics improvements, implementation examples